M256 Unit 1
UNDERGRADUATE COMPUTING

Software development with Java

Introduction to software development

Unit 1

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenq@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1557 6

1.1

CONTENTS

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Compositor

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

Welcome to M256 *Software development with Java*.

By the time you begin this course you will have written a fair number of programs in Java. When you are writing code you naturally concentrate on how to make the code do what is required. But how do you know what is required? Where does this information come from? Consider the types of question you might ask before starting to write code, for example:

See the *Course Guide* for more detailed information on the prerequisite Java knowledge you will need for this course.

- What should the program do?
- What classes will the program use?
- What information must objects record, and to what messages should they respond?
- How will the user interact with the program?
- How can you test that the program works as it is supposed to?
- How easy will it be to adapt the program if things change later?

1.1 Software development

Software development, the process of getting from a customer's needs to operational software that meets those needs, involves finding answers to the questions above and to a range of related questions. Notice the term *development* – this implies something that emerges gradually, as part of a process, and does not happen all at once. Much of software development is about planning the software, and this involves building models described by text or diagrams, or both. You will probably have met some kinds of model already; for example, programs are often modelled with flow diagrams, or with structured English.

If you were writing a very small program you might not need to do much, if any, planning. For example, a program to accept two integers and output the larger might be something that you could write by typing the code straight into the computer and fixing any problems as you went along. You would obviously have to think ahead a bit, but you would not need to start by developing any models. If you did it would probably just slow you down without contributing anything.

But this only applies to very simple cases. After all, finding the larger of two numbers is not that useful, since this is something you could easily do without a computer. Programs which do really useful things are going to be more complicated. So, consider the following imaginary example. The description is simplified – a similar real-world program would be considerably more complex – and software that performs these tasks has been around for a long time in any case. Nevertheless the example will be helpful as an introduction.

1.2 Complex software systems

You have a friend who works in the administration of a local school and wants some software to help her do her job. She gives you the following information.

- The secretary keeps a record of each pupil's name and date of birth.
- All pupils are aged between 4 and 18 inclusive.

- Each teacher's name is recorded.
- Each form has a name (e.g. 'Form 1b'), contains up to 10 pupils and is taught by a single teacher.

Here the term *form* is being used as a synonym for *class*, that is, a form is a small group of pupils (which, in this particular situation, are allocated to a single teacher).

Some software is needed that will support the secretary in fulfilling the following tasks.

- For a given form, list information about its pupils and its teacher.
- Record the enrolment of a new pupil into a form.
- Provide the name of the teacher with the most pupils in their form.
- Provide the name of the oldest pupil in a form.

Naturally you want to help your friend, but how will you start? Of course you might try to write the program as you went along, as for the simple program considered previously. Unfortunately this approach will not work any longer. True, you have a description of what the program has to do, however there are many questions that must be answered before any code can be written. For example:

- What classes will you use?
- What Java class libraries are needed?
- What instance variables and methods will you define for any new classes?
- How are the classes related to one another?
- When the program runs, what objects will exist, and how will they be created?
- What messages will be sent, and to what objects?

You might be able to make a stab at the answers to some of these, but we hope you can see that, even with this relatively simple example, there is almost no chance of writing an operational system without doing some careful planning. Now imagine you were dealing with a complex system such as the following.

Iridium satellite system

In 2001 the Iridium satellite system, initiated by Motorola and managed by Boeing, became operational; the culmination of several years of software development. Iridium is a satellite-based communication system enabling wireless communication (for example, using mobile phones and pagers) around the world, even in remote areas. Software was developed to enable communication between mobile phones and land-based communication lines via sixty-six low-orbit satellites. This project, involving object-oriented software development processes and programming languages, produced more than 15 million lines of code.

Not only was the amount of planning for Iridium huge, but it is impossible to imagine a single programmer being able to create the system – in fact hundreds of software developers were involved. This raises another set of issues. How can a team of people succeed in working collaboratively on complex projects? How can their activities be coordinated? How do they communicate with one another?

1.3 The aims of M256 and this unit

In M256 you will be introduced to software development activities that help individuals and teams create complex object-oriented software which meets its users' requirements. You will acquire skills, and learn about concepts and techniques that will be valuable when you create programs, as well as giving you an understanding of how software development is carried out on a large scale by teams of professionals.

By 'professionals' we simply mean people who develop software for a living.

Throughout M256 small, relatively simple systems will be developed, so you will able to appreciate how the stages of development fit together. However, you will be learning about activities that can be scaled up to very much larger and more complex systems.

These explorations will take you from analysing an initial description of what is required of a system (the **requirements**) to implementing the system in Java (that is, writing the Java code). You will learn how to develop software in a series of linked stages, ending up with a working system – although you will see that even when the code is written the job is by no means finished!

You can expect to develop both your Java programming skills and your understanding of object-oriented concepts. However, it is not intended that M256 will introduce you to any significant new Java or object-oriented features, rather it will enable you to apply and extend your object-oriented programming experience to the development of software systems of increasing interest and complexity. You will, through exercises (some of which have practical elements) participate in the development of the systems used as examples in this course. Active participation is *essential*; it is impossible to acquire the skills of software development just by reading. You need to gain practical experience by actually carrying out the tasks involved. The exercise discussions are a key part of the teaching material. We strongly advise you to read them as you complete each exercise, because they often contain important teaching points.

This first unit of M256 aims to:

- remind you about some fundamental Java and object-oriented concepts (Sections 2 and 3);
- give you practice in using the course software (Sections 2 and 3);
- introduce object diagrams and sequence diagrams (Sections 2 and 3);
- preview the software development phases you will learn about during the course (Sections 4 and 5);
- explain the ideas behind software development (Sections 4, 5 and 6).

Please note that *Unit 1* will be familiar to you if you have studied M255, since it revises and expands upon the introduction to software development given in *Unit 14* of that course. In particular, you will be looking here at an almost identical example system, although in this unit you will explore it in more depth, and from different perspectives, and you will use a different environment.

1.4 Terminology

Before concluding this introduction, we will clarify the terminology we use for the different kinds of program you will meet in this course.

A **Java application** is a program run directly by the Java Virtual Machine (JVM) and which requires a `main()` method. This may be the only kind of program you have experience of, and it is programs of this kind that are created in M256. Such a program can be run from within an integrated development environment (IDE), a special piece of software used both to create and execute programs. However, users of the program do not need to have an IDE – all they need is a suitable Java Runtime Environment (JRE), something that is freely available for all the main software platforms.

If you have been working within a customised workspace environment you may not have always been aware of working with a `main()` method.

More generally, an **application** is a program that performs a specific function directly for the user. This is in contrast to software such as an operating system which exists to support applications. In effect, an application turns your computer into a specialised computer, such as a word processor or web browser.

We use the term **software system** (sometimes just **system** or **software**) to indicate a program which is large in the sense that it carries out a number of tasks, some of which may be complex. M256 is concerned with developing software systems. A software system is a dynamic entity which, in an object-oriented context, is comprised of interacting objects. The **system code** is the Java code for the system – the source files which, when compiled and run, generate the system.

1.5 Studying this unit

As well as exercises involving practical elements, this unit also contains a number of self-assessment questions (SAQs) and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on. Some of the exercises require you to sketch diagrams, which you may wish to do either by hand or by using a drawing package.

In Sections 2 and 3 you are directed to the software for the course, with which you are required to explore a Java system. You are likely to find that Sections 2 and 3 each require more time than the later sections.

Sections 4, 5 and 6 involve no practical work. We expect that each of these sections will require approximately the same study time.

2 Exploring objects

In this section you will:

- use the course software to investigate part of an implementation of the school administration system introduced in the previous section;
- review some object-oriented concepts that are particularly important in the rest of this course;
- be introduced to the use of object diagrams to illustrate the structure of objects and their interrelationships.

2.1 The IDE

To compile a Java program a Java Software Development Kit (SDK) is commonly used. It is perfectly possible to write and run Java programs simply using the appropriate SDK for the Java version you are using, however, many developers choose to work within an **integrated development environment (IDE)**. An IDE is a software tool which facilitates many of the tasks associated with writing and running programs in a specific language. Such a tool may, along with other facilities, include the following.

- A specialised editor for writing and editing source code.
- Facilities for checking syntax.
- Facilities for structuring programs into separate projects, and for creating repositories of associated documents.
- An integrated compiler and interpreter.
- Facilities for 'stepping through' code as it is executed, to explore the changing states of objects, for example.

In M256 you will use the NetBeans IDE from Sun Microsystems both for exploring examples of code and, in later units, for implementing the systems that are developed.

Note that there will be several units in which NetBeans is not used at all.

Section 1 included a description of the requirements for a system to help in the administration of a school. We, the M256 course team, have developed such a system, and you will explore it in the following exercise. We will call this software the School System.

Exercise 1

If you have not yet installed the course code files and NetBeans, and worked through the Introduction, Section 2 and Subsections 3.1 and 3.2 of the *NetBeans Guide*, as directed in the *Course Guide*, do so now. This will prepare you for the practical work to be undertaken in this exercise (by, amongst other things, showing you how to compile and run Java projects).

Launch NetBeans and open the project called `School`, which, if you installed the course code files as directed, should be located on your computer in the folder `My Documents\M256\M256Code\Systems`. Compile and run it. You should be presented with the graphical user interface shown in Figure 1.

Compiling a project (i.e. compiling the Java files it contains) is referred to in NetBeans as 'building' the project.

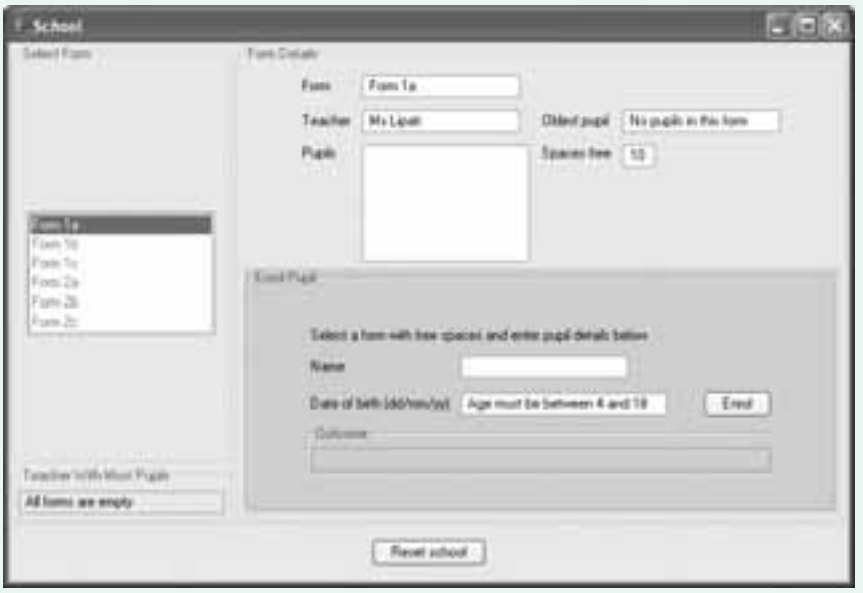


Figure 1 A simple graphical user interface for the School System

(a) Using the user interface, carry out the following tasks.

(i) View the details of each form.

(ii) Enrol, into the form named Form 1b, pupils with the following details:

Rosie Webster, who has the date of birth: 24/12/00.

Chesney Brown, who has the date of birth: 05/05/01.

David Platt, who has the date of birth: 25/12/00.

Enrol, into Form 1c, the following pupil:

Sophie Webster, who has the date of birth: 04/11/00.

What do you see in the user interface as a result of your actions? View the details of Forms 1b and 1c.

(iii) Try to enrol pupils with the following details into Form 1a:

Vera Duckworth, who has the date of birth: 12/03/45.

Joshua Peacock, who has the date of birth: 08/04/19.

Note that the software interprets the year 45 as 1945 and 19 as 2019.

What occurs in the user interface as a result of your actions?

Do not enrol any more pupils into the school yet. If you do so accidentally, or if you make a mistake and enter incorrect details, click on the `Reset school` button to restore the system to its pristine initial state. For simplicity, this basic system allows no other way of correcting errors.

Close the user interface.

(b) Now look at how the project source code is structured. It is in two parts called `schoolgui` and `schoolcore`. These are the two packages that make up the School System.

Look briefly at the classes contained by each package. There is no expectation that you will understand all the details.

Later in the course we will discuss the concept of a package in more detail. For now, you just need to know that it is a way of grouping together related classes.

(c) Which classes in the package `schoolcore` would you expect to have instances corresponding to real-world entities (i.e. 'things' in the real world)?

You can, if you wish, close NetBeans at this point; the state of the School System will automatically be saved.

Discussion

(a) The widgets in the user interface should be familiar and, hopefully, the tasks were straightforward.

(i) To view the details of each form, select the form's name in the list of forms. As well as the form's name you should have found that the following are initially displayed:

- the name of the form's teacher;
- a message informing you that there are no pupils in the form;
- the number of spaces left in the form.

Initially each form is empty, so they each have 10 spaces.

Recall that a form contains up to 10 pupils.

(ii) Enrolling a pupil involves selecting the form's name, entering the pupil's name and date of birth into the relevant fields and clicking on the `Enrol` button.

Each time, a pupil is enrolled a message of the form `Pupil enrolled (age x)` – where `x` is the age of the pupil in the current year – is displayed in both an `Information` dialogue box, and in the `Outcome` field. When the pupil has been successfully enrolled the `Name` and `Date of birth` fields are cleared.

Selecting each form's name in turn should reveal that Form 1b and Form 1c have pupils in them, whose names and dates of birth are displayed. The name of the oldest pupil in each form is also displayed, and the number of spaces in each form is adjusted accordingly. The other forms remain empty. The name of the teacher with the most pupils is displayed.

(iii) An attempt to enrol the pupil named Vera Duckworth with the date of birth 12/03/45 results in a `Warning` dialogue box with the message `Pupil too old! (age y)`, where `y` is Vera's age in the current year. The error message is also displayed in the `Outcome` field. The `Name` and `Date of birth` fields are not cleared.

Recall the restriction that the school only accepts pupils aged between 4 and 18.

An attempt to enrol the pupil named Joshua Peacock with date of birth 08/04/19 results in the warning, `Pupil too young! (age -1)`.

You can think of –1 as an error code.

(b) `schoolgui` contains a single class, `SchoolGUI`. This contains the code implementing the graphical user interface and includes a `main()` method, the required entry point into a Java program, which creates a `SchoolGUI` object.

Much of the code for this class has been automatically generated using a facility provided by NetBeans. You will learn how to work with this facility later in the course; at this point you do not need to understand the code.

`schoolcore` contains the following classes: `Form` , `Pupil`, `SchoolCoord`, and `Teacher`. These classes are discussed in (c) below.

You may notice the use of the class `M256Date`, from the package `m256date`. This class is provided to simplify your work with dates throughout the course, relieving you of some of the complexity of using the predefined Java date classes.

(c) `Form`, `Teacher`, and `Pupil` objects correspond to real-world forms, teachers and pupils respectively. You can deduce this from the names of these classes, and from the class comments. For example the following comment immediately precedes the `Pupil` class definition.

```
/**
 * Represents pupils, each having a name and a date of birth.
 */
```

An instance of the class `SchoolCoord` does not correspond directly with a real-world entity but is used to handle communication with the user interface. You will learn more about the specific role of such objects as the course progresses.

It is common in object-oriented systems to consider the user interface as distinct from the rest of the system. This is why there is a separate package, `schoolgui`, for the user interface in the School System code. There are many benefits to this separation, which we discuss later in the course. In M256 we refer to the 'rest of the system' as the **core system**. This is the part of the software that usually contains objects with real-world equivalents – `Pupil` objects, for example. The package `schoolcore` contains the core School System code.

Other names for the core system include *domain model* and *business model*.

2.2 Object diagrams

You should be very familiar with the fact that at the heart of object-oriented software is the concept of an **object**, which consists of a collection of data and a set of operations that can be applied to the data. The kind of data an object holds, that is its attributes (generally implemented in Java as instance variables), and its operations (implemented as methods) are determined by the object's class – objects of the same **class** having the same attributes and operations.

One of the great benefits of object-orientation is the frequent correspondence of objects with real-world entities, as illustrated in Exercise 1. For example, `Pupil` objects correspond to real people who are pupils in the school. They each have a `String` instance variable, `name`, and an `M256Date` instance variable, `birthDate`. Consequently, when a pupil with the name Rosie Webster and the date of birth 24/12/00 enrols in the school, you might recognise intuitively that this should be mirrored by the creation of a `Pupil` object with corresponding attribute values (which define its **state**). That is, the `Pupil` object's `name` instance variable references the `String` object `"Rosie Webster"` and its `birthDate` instance variable references the `M256Date` object that represents the date 24/12/00. We can illustrate this `Pupil` object in the following **object diagram**.

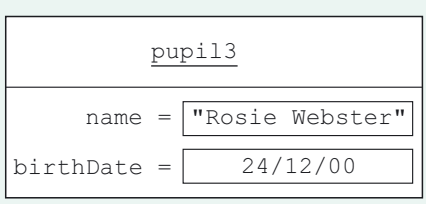


Figure 2 Object diagram

In an object diagram, objects are represented by rectangles. The text in the upper section of an object rectangle shows the chosen means of identifying the object. Thus we have called the `Pupil` object in Figure 2, `pupil3`. Note that `pupil3` is just a label – an **identifier** – that is used to refer to the object involved in discussions and diagrams. It allows this particular `Pupil` object to be distinguished from other `Pupil` objects in the system. It is *not* intended to be a variable name and, in this example in particular, it should not be confused with the value of the object's `name` instance variable, which is a `String` representing the name of the pupil (`"Rosie Webster"`). You are free to choose any text you like as the identifier for an object, so long as it clearly indicates the class of the object, and is different from other identifiers already in use.

For core system classes that correspond to real-world concepts M256 uses the convention that identifiers will consist of the lower-case version of the class name, augmented with a number (e.g. `pupil1`, `form2`, etc.). However, for other classes more distinctive identifiers have been used, such as `school` (for a `SchoolCoord` object) and `userInterface` (for a `SchoolGUI` object).

The lower section of an object rectangle shows the attribute values of the object. This section may be omitted if the attribute values are not of interest.

Object diagrams can illustrate connections, or **links** between objects, mirroring the connections between their real-world equivalents. We can illustrate the fact that Rosie Webster is enrolled into the form named Form 1b, which is taught by Mr Barlow, in the following way.

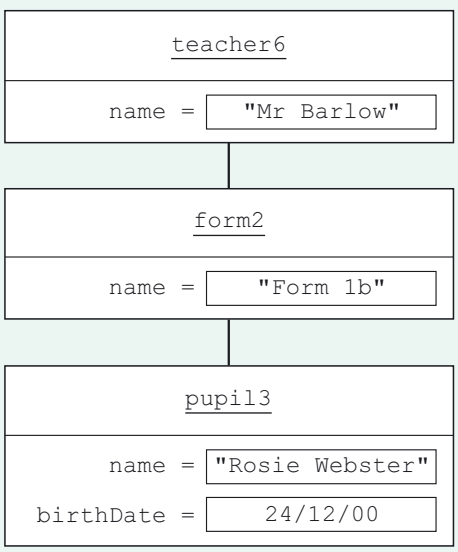


Figure 3 Object diagram illustrating links

The lines running between the object rectangles in Figure 3 illustrate links between the objects. Thus the line between the `form2` rectangle and the `pupil3` rectangle illustrates a link between `form2` and `pupil3`, and represents the fact that the form (Form 1b) corresponding to the object `form2` has in it the pupil (Rosie Webster) corresponding to the object `pupil3`.

SAQ 1

In Figure 3, what does the line between the `teacher6` rectangle and the `form2` rectangle illustrate?

ANSWER.....

It illustrates a link between the objects `teacher6` and `form2`, representing the fact that the teacher corresponding to `teacher6` (that is, Mr Barlow) teaches the form corresponding to `form2` (that is, Form 1b).

The diagram in Figure 3 shows only part of the School System – it is a partial snapshot at a particular point in time. The full running system would contain many more objects and links between them, depending on the pupils, teachers and forms in the school at that time. In an object diagram we need only include those objects that we are interested in. For example, although the diagram shows only one `Pupil` object, there may well be other pupils in the form we have called `form2`. We refer to the full complement of objects, their attribute values (that is, the objects' states) and the links between them, which constitute the system at any one time, as the **state** of the system at that time.

Exercise 2

Extend the object diagram in Figure 3 to show that the pupils Chesney Brown and David Platt (who you enrolled into the school in Exercise 1) are also in the form represented by `form2`, whose teacher is represented by `teacher6`.

Discussion

Figure 4 shows the extended object diagram – you may have used different identifiers for the `Pupil` objects.

Figure 4 Object diagram illustrating `teacher6`, `form2` and its `Pupil` objects

This diagram is hand-drawn to emphasise that, for many purposes, and certainly for answering the exercises in the course units, this means of creating diagrams is perfectly acceptable. Real software developers scribble plenty of diagrams on whiteboards, on the backs of envelopes, etc. From time to time we will do the same again to reinforce this point. If you prefer to use a drawing tool for the exercises then, of course, do so.

An object diagram does not imply anything about *how* links between objects are implemented, just that some connection exists. There are different ways to implement links, and in the following exercise you will see how this has been done in the School System.

Exercise 3

In NetBeans return to the package `schoolcore` within the `School` project. You will not actually be doing anything with the system, just thinking about what it contains.

(a) The running system contains, amongst other objects, the `Teacher` and `Form` objects illustrated in Figure 4. That is, there is a `Teacher` object with its `name` instance variable set to `"Mr Barlow"` and a `Form` object with its `name` instance variable set to `"Form 1b"`. Of course, there is nothing in the system that mentions the identifiers we used (`teacher6` and `form2`); remember that these are just labels used in an object diagram (which is external to the system).

Look at the source code for the classes `Teacher` and `Form`, in particular the instance variable declarations. How is the link between `teacher6` and `form2` implemented?

(b) If you followed the instructions in Exercise 1 and enrolled three pupils into Form 1b, then in the running system the `Form` object we are referring to as `form2` is linked to three different `Pupil` objects, as illustrated in Figure 4. How are these links implemented?

Discussion

(a) Although objects in the running system cannot be 'seen', they are generated from the source code from which you can glean information about them. The following variable declaration in the `Form` class is the key here.

```
private Teacher teacher;    /** the teacher of the form*/
```

This shows that the link is implemented by `form2` having an instance variable, `teacher`, which references `teacher6`.

This situation is not unique to `form2`, of course; *every* `Form` object has a reference to the relevant `Teacher` object. You should note that a `Teacher` object does not hold a reference to the relevant `Form` object – there is no corresponding `Form` variable declaration in the `Teacher` class.

(b) Here is the relevant declaration, again in the `Form` class.

```
private Collection<Pupil> pupils;    /**
                                      *a collection of the pupils in the form
                                      */
```

This shows that the links are implemented by a `Form` object (here, `form2`) having an instance variable, `pupils`, which references a `Collection` of the `Pupil` objects that represent pupils in the form.

In fact you can see from the following code within the `Form` constructor that, when the code is run, `pupils` actually references an instance of `HashSet` that contains `Pupil` objects:

```
pupils = new HashSet<Pupil>();
```

Note that a `Pupil` object has no reference to the linked `Form` object.

Recall that `HashSet` is one of the Java `Collection` classes.

Also note that in this course, our coding convention is *not* to use the Java keyword `this`, which you may have seen in other courses, as in the following alternative code:

```
this.pupils =
new HashSet<Pupil>();
```

Links between objects may be implemented by instance variables in both classes, or in just one class as is the case in our examples above. The choice of which implementation is appropriate depends on the use that the system makes of the links, as you will learn later in the course.

Note that, although both attributes and links can be implemented using instance variables, they are represented very differently in an object diagram. This representation highlights the fact that an object's attribute values are simple pieces of information (represented by strings, for example) that are not specific to the system under consideration, whilst in contrast its links are with other core system objects.

In this section, you have used NetBeans to run a small software system and learnt about object diagrams. In the process, you have reviewed important concepts such as the correspondence between objects and real-world entities; and you have met ideas such as links between objects, and the partitioning of a system into a user interface and a core system

In the next section, you will continue to review important object-oriented concepts as you learn how to model interactions between objects.

3 Exploring object interactions

In Section 2 you inspected part of the School System code, and used it to reflect on objects in the system. In this section you will:

- build on this work in revising how objects interact to achieve tasks;
- learn how to illustrate object interactions using sequence diagrams.

3.1 Collaborating objects

A key idea in object-oriented software is that of tasks being carried out by objects that request services from one another by sending **messages**. When an object receives a message its corresponding method is **invoked** or **called**; that is, the method code is executed.

Within a system each object can be thought of as having certain responsibilities in meeting a particular part of the system's overall **behaviour** (the set of tasks required of the system). Thus the behaviour of the whole system is derived from the interaction or **collaboration** between individual objects. Collaboration involves one object requesting a service (via a message) from another object to help it fulfil a certain responsibility.

To explore this fundamental idea consider the requirement of the School System to provide the name of the oldest pupil in a form. In Exercise 1 you enrolled the following pupils into the form called Form 1b:

Rosie Webster, date of birth: 24/12/00.

Chesney Brown, date of birth: 05/05/01.

David Platt, date of birth: 25/12/00.

This particular scenario is illustrated in the object diagram in Figure 5, which shows the `Form` object `form2` (named Form 1b) together with the `Pupil` objects corresponding to all the above pupils in the form, which were referred to as `pupil3`, `pupil4` and `pupil5` in Figure 4. The system of course includes other `Form` and `Pupil` objects, as well as objects of other classes, but they are not relevant to our current investigations.

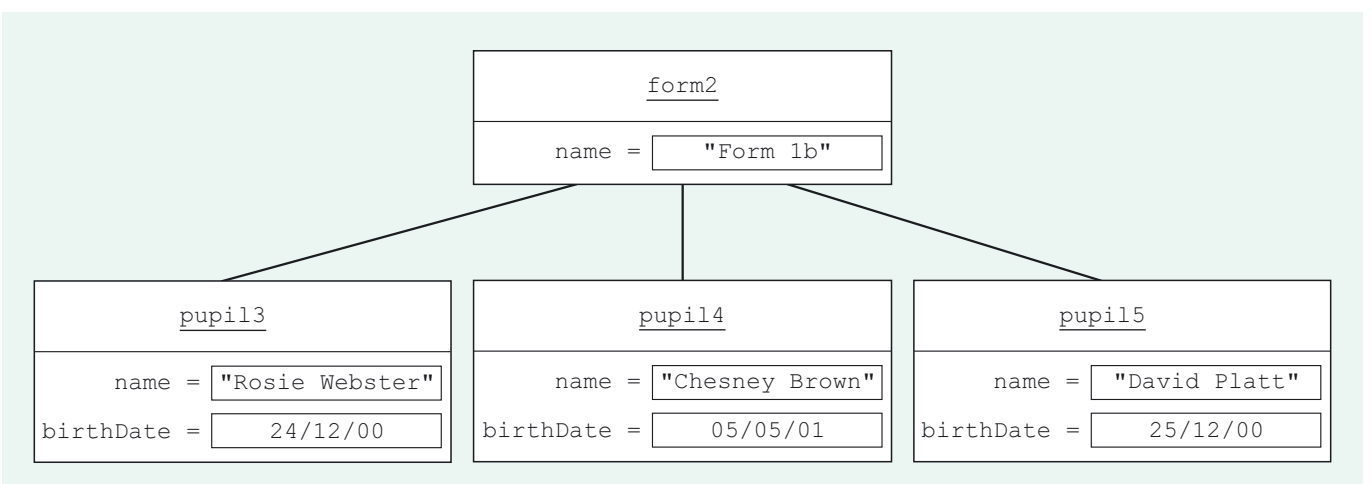
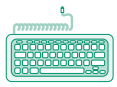


Figure 5 `form2` and its `Pupil` objects

Exercise 4



In NetBeans run the `School` project.

(a) In the user interface, select `Form 1b`. What is displayed in the field labelled `Oldest pupil`?

(b) Select `Form 2a`. What is displayed in the field labelled `Oldest pupil`?

(c) Now turn to the source code, and the package `schoolcore`.

Read the comment for the method `getOldestPupil(aForm)` in class `SchoolCoord` (that is, the method with the header `public Pupil getOldestPupil(Form aForm)`). Suppose that `school` is a `SchoolCoord` object and `form2` is the `Form` object representing the form Form 1b. What is the result of sending `school` the message `getOldestPupil(form2)`?

What would be returned if the form were empty?

(d) Read the comment for the method `getOldestPupil()` in class `Form`. What is the result of sending `form2` the message `getOldestPupil()`?

What would be returned if the form were empty?

Note that here we are using `school` as an identifier for a `SchoolCoord` object, and `form2` as an identifier for a `Form` object. In fact, in the `SchoolGUI` code, there also happens to be a variable called `school` referencing a `SchoolCoord` object, but bear in mind that identifiers do not necessarily correspond to variable names.

Discussion

(a) The name `Rosie Webster` is displayed.

(b) The text `No pupils in this form` is displayed.

(c) `school`'s method `getOldestPupil(aForm)` is invoked with `form2` as the actual argument. The `Pupil` object corresponding to the oldest pupil in Form 1b is returned. The oldest pupil in Form 1b is Rosie Webster, who corresponds to the object we are calling `pupil3`; so the `Pupil` object `pupil3` is returned.

`null` would be returned if the form were empty.

(d) `form2`'s method `getOldestPupil()` is invoked. The `Pupil` object corresponding to the oldest pupil in the form Form 1b is returned. As above, this is the `Pupil` object `pupil3`.

`null` would be returned if the form were empty.

Now consider what happened when, in carrying out Exercise 4, you selected `Form 1b`. Selecting a form name results in the name of the oldest pupil in that form being displayed in the user interface (or a report that there are no pupils). But what happens within the system to achieve this? You saw in Exercise 4 that `SchoolCoord` and `Form` have relevant methods, but when and how are these invoked?

We are interested here not in how the user interface works, but in the interactions within the core system.

When a form name is selected a sequence of messages is initiated between objects in the system as the objects collaborate to provide the information required to the user interface. The sequence we are interested in starts with the user interface sending the message `getOldestPupil(form2)` to a `SchoolCoord` object called `school`. You will need to take our word for this as we do not want to consider the user interface code here. It suffices to note that the user interface is implemented as an instance of the class `SchoolGUI`, which we will call `userInterface`.

You will learn later in the course about the special role of the class `SchoolCoord`. For now, you only need to know that there is only one `SchoolCoord` object, called `school`, whose job it is to receive messages from the user interface and return information to it.

A collaboration is underway. `userInterface` requests that `school` gets the oldest pupil in the form. We call `school` a **collaborator** for `userInterface` in carrying out the task of finding the oldest pupil. An alternative terminology calls `school` the **server** and `userInterface` the **client** for this collaboration. The client sends the message to the server to request a service and in response the server provides the service to the client.

3.2 Sequence diagrams

There are several powerful diagrammatic ways to illustrate the message passing involved in specific collaborations. Throughout M256 we will use **sequence diagrams**. A sequence diagram is quite different from an object diagram, although they both illustrate objects. An object diagram shows the state of part of the system at a particular point in time and, as such, can be described as a **static model**. A sequence diagram shows objects interacting by sending messages to each other to carry out a particular task. As it illustrates events occurring in the system over time, a sequence diagram is classed as a **dynamic model**.

In Exercise 4 you studied how certain objects in the School System collaborated to find the oldest pupil in Form 1b. First, the message `getOldestPupil(form2)` was sent from `userInterface` to a `SchoolCoord` object called `school`. Here is a sequence diagram showing this first step taking place.

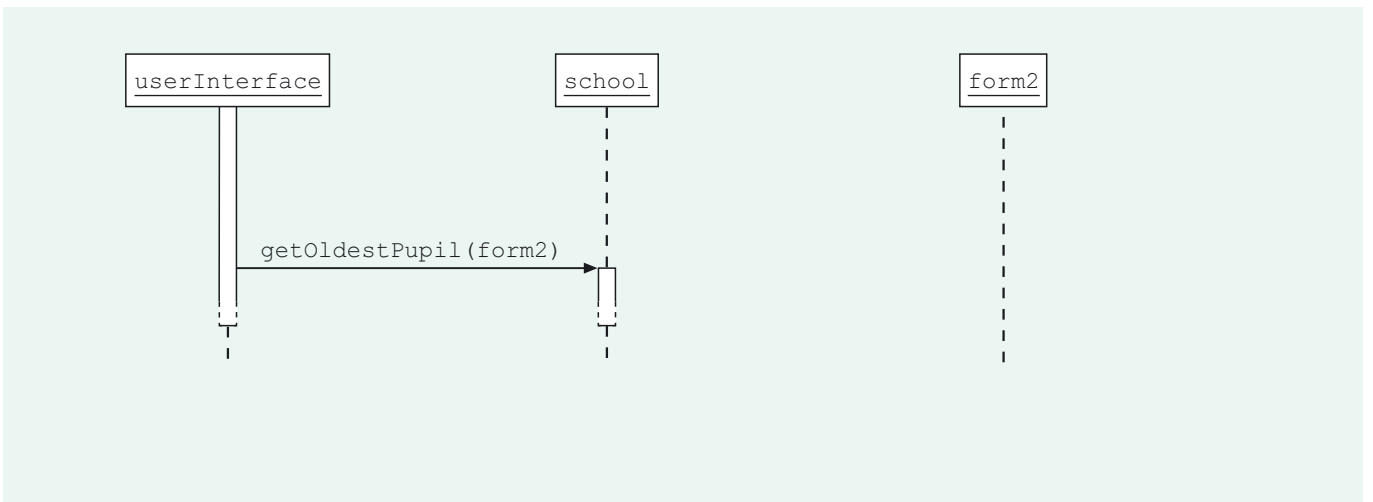


Figure 6 A simple sequence diagram

Annotating a sequence diagram can be a useful way of explaining more about some aspect of the situation being illustrated.

This sequence diagram might appear simple, but take time to identify what each part of it represents, as this will be vital in understanding more complex sequence diagrams. Figure 7 is an annotated version of this sequence diagram. Study it in conjunction with the subsequent points.

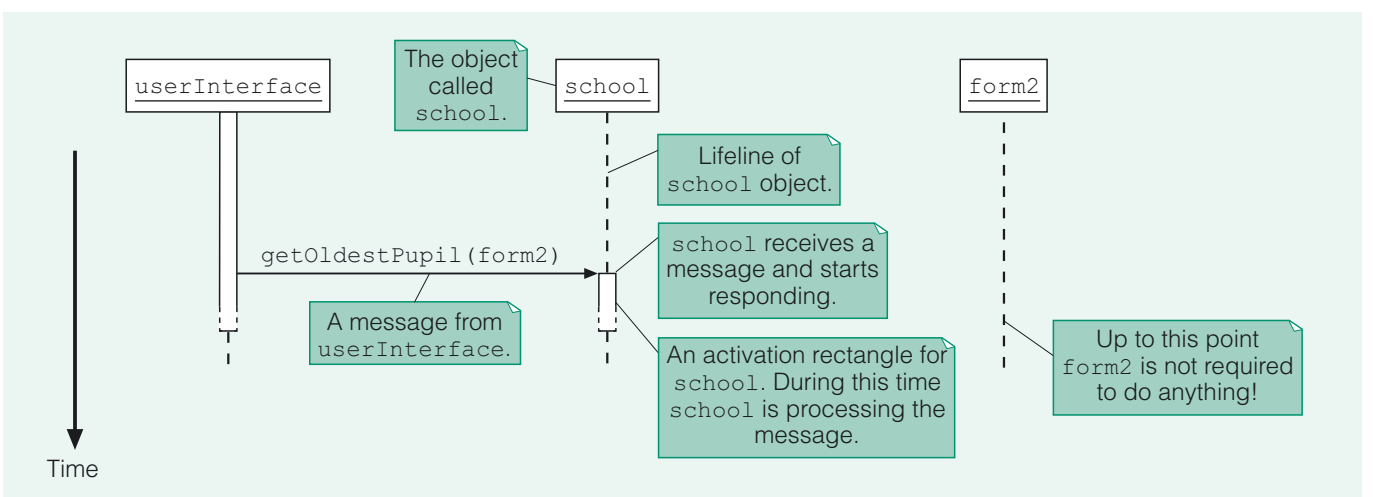


Figure 7 Simple sequence diagram, annotated

Here are some important features to note about sequence diagrams.

- Each object is represented by a rectangle, just as in an object diagram. This rectangle contains an identifier for the object, but no attribute values.
- Time is viewed as running vertically downwards. This will be more meaningful when we consider more elaborate sequence diagrams – this will be discussed further below.
- A dashed vertical line running down from an object rectangle represents the **lifeline** of that object, that is, the time during which the object exists.
- When an object receives a message, an **activation rectangle** running vertically downwards is started on that object's lifeline. This represents the period during which the object is engaged in responding to the message it has received; that is, the time during which the method invoked by the message is being executed.
- An 'endless' activation rectangle (i.e. where the bottom of the rectangle is dashed) indicates that the object has not completed its processing. Similarly, a dashed top on an activation rectangle implies the object has previously been involved with processing that is not depicted on the diagram.
- The activation rectangle for the `userInterface` object comes straight out of the object rectangle and always appears endless. This indicates that the user interface is continuously active, always listening for events (mouse clicks, for example) caused by the user.
- A message is represented as a solid arrow.

When `school` receives the message `getOldestPupil(form2)`, its method `getOldestPupil(aForm)` is invoked with `form2` as argument. What happens next? To find out, look at the method code.

```
public Pupil getOldestPupil(Form aForm)
{
    return aForm.getOldestPupil();
}
```

From the method code you can see that executing the method involves several things. Firstly, `school` sends the message `getOldestPupil()` to `form2`. That is, `school` enlists the collaboration of `form2`. To show this on the sequence diagram we need to draw a message arrow coming from the activation rectangle of `school` and going to `form2` as shown in Figure 8.

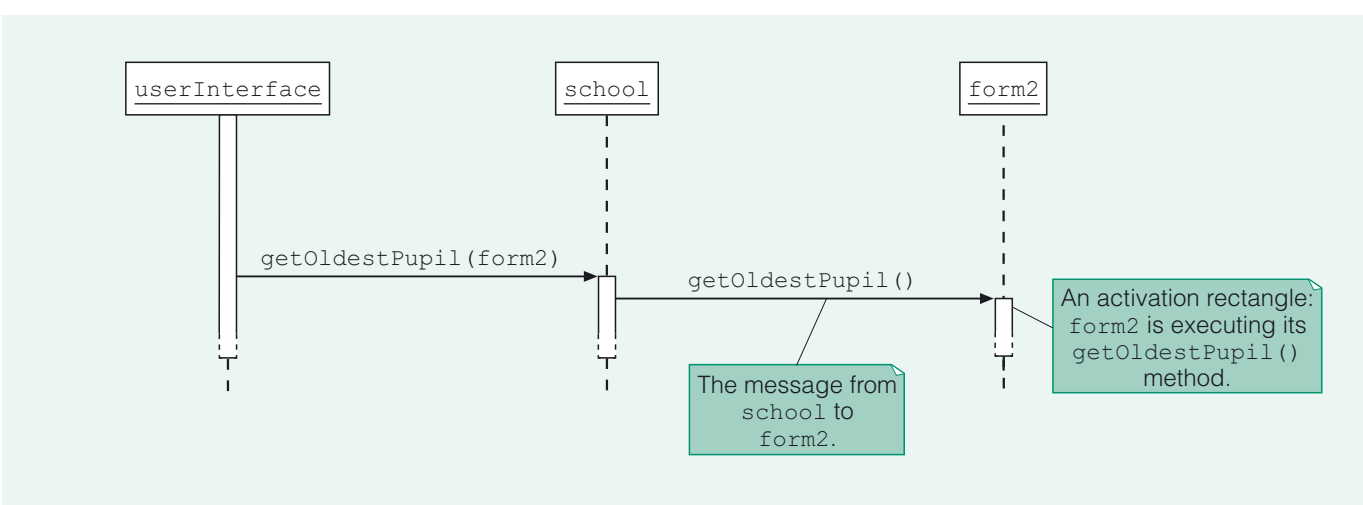


Figure 8 `school` sends a message

Do not confuse the two methods involved. The first is `getOldestPupil(aForm)` in class `SchoolCoord` and the second is `getOldestPupil()` in class `Form`. Although they happen to have the same names they are entirely distinct.

Note that the second message arrow in Figure 8, representing the message `getOldestPupil()` sent to `form2`, starts from `school`'s activation rectangle, a little way down the page from the point at which `school` receives its message. This illustrates that `school` *first* receives the message `getOldestPupil(form2)`, *then* sends the message `getOldestPupil()` to `form2`. Thus the passage of time is illustrated moving down the diagram.

Secondly, when `form2` receives the message `getOldestPupil()`, it executes its `getOldestPupil()` method code. You will shortly study in more detail what this involves; for now, the focus is on the communication between `form2` and `school`. In Figure 9, `form2` is shown returning a message answer to `school`, which in our current system is the object `pupil3`, corresponding to Rosie Webster, the oldest pupil in the form.

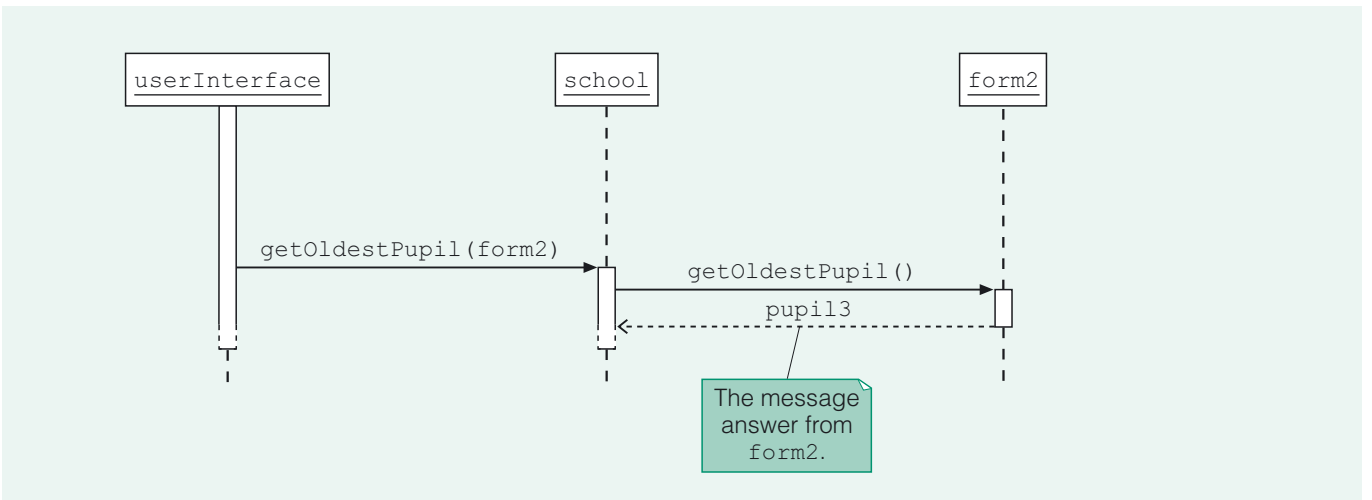


Figure 9 `form2` returns a message answer

Strictly speaking it is not an *object* (e.g. `pupil3`) that is returned by a message answer, but a *reference to* an object.

Figure 9 depicts the message answer, `pupil3`, being returned by the `Form` method `getOldestPupil()`. A message answer is shown as a labelled dashed arrow emanating from the bottom of an activation rectangle, since it occurs when the object completes the processing represented by the activation rectangle. Note that a message answer is of course *not* a message (i.e. it does not invoke a method); it is simply an object (or in other examples it could be a value of a primitive data type, such as `int`) that may be returned (to the client) as an object's final response to receiving a message. Message answers may be omitted from a sequence diagram if there is no need to emphasise them and, of course, not all messages yield a corresponding message answer.

Finally, as shown in Figure 10, `school` returns (to `userInterface`) `pupil3` as its response to the invocation of its method `getOldestPupil(aForm)`.

At this point `userInterface` retrieves the `name` and `birthDate` of `pupil3` in order to display it to the user. Hence there is a subsequent collaboration between `userInterface` and `pupil3` that we are not considering here.

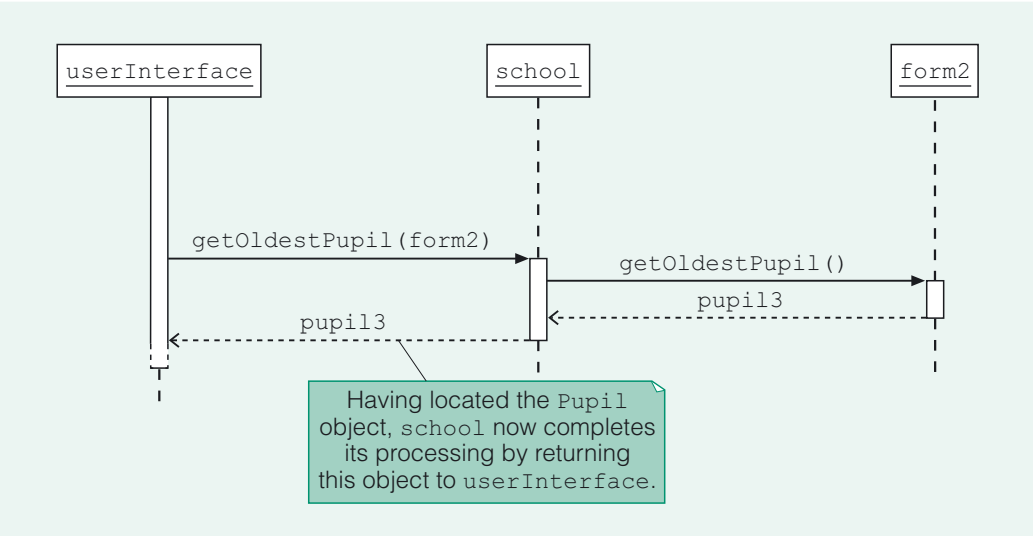


Figure 10 `school` returns a message answer

The above sequence diagram illustrates the collaborations between `userInterface`, `school` and `form2` when the system retrieves the oldest `Pupil` object from `form2`. It is important to note that the sequence diagram does not, by itself, impart anything about the details of the code involved other than the messages that are sent, and their responses. It is an **abstraction**, omitting some details (the actual code that is executed, for example) and stressing others (the collaborations).

SAQ 2

For the collaboration between `school` and `form2` illustrated in Figure 10, which object is the client and which is the server?

ANSWER.....

In the collaboration shown in Figure 10, `school` is the client and `form2` is the server.

Although `school` is a client to `form2`, recall that, as described at the end of Subsection 3.1, `school` is also a server to `userInterface`. This is a familiar pattern with collaborating objects: the same object can be a server to one object and a client to another.

3.3 Creating sequence diagrams

In considering how the system determines the oldest pupil in a form, we have looked at collaborations between `userInterface`, `school` and `form2`. But `form2` participates in some additional collaborations that we have yet to think about.

What is involved in `form2` locating and returning the `Pupil` object corresponding to the oldest pupil in the form, when it receives the message `getOldestPupil()`? Well, on receiving the message, `form2`'s `getOldestPupil()` method is invoked. Here is the method.

Note, this method has package accessibility – this will be discussed in *Units 5* and *7*.

In M256, we generally do not use a class's accessor methods inside that class. Hence here, for example, we have simply `pupils`, and not `getPupils()`.

```
Pupil getOldestPupil()
    {
        M256Date birthDate;
            //set firstBirthDate to today's date
        M256Date firstBirthDate = new M256Date();
        Pupil oldestPupil = null;
            //iterate through the receiver's pupils
        for (Pupil p : pupils)
        {
            birthDate = p.getBirthDate();
                //if this is the oldest pupil so far...
            if (birthDate.before(firstBirthDate))
            {
                //...set firstBirthDate to this pupil's birth date...
                firstBirthDate = birthDate;
                    //...and set oldestPupil to this pupil.
                    oldestPupil = p;
            }
        }
        return oldestPupil;
    }
```

Exercise 5

(a) Based on the above code, and recalling that each `Form` object has an instance variable `pupils` that references a collection of `Pupil` objects, outline how, when `form2` receives a `getOldestPupil()` message, it locates and returns the `Pupil` object corresponding to the oldest pupil.

(b) Identify an example of collaboration that occurs as a *result* of a `getOldestPupil()` message being received by `form2`.

Discussion

(a) When `form2` receives a `getOldestPupil()` message its `getOldestPupil()` method is invoked. During the subsequent processing `form2` iterates over all the `Pupil` objects in its `pupils` collection, sending each in turn the message `getBirthDate()`. It compares each birth date with the earliest one found so far (held in the local variable `firstBirthDate`) and sets the local variable `oldestPupil` to reference the oldest `Pupil` object found so far. Finally, once all `Pupil` objects have been interrogated, it returns the overall oldest `Pupil` object, `pupil3` in this scenario, as the message answer.

(b) Each `Pupil` object in `form2`'s `pupils` collection collaborates (as a server) with `form2`. The `Pupil` object provides its birth date, in response to a `getBirthDate()` message sent from `form2`.

In Exercise 5 you saw that `form2` collaborates with its `Pupil` objects. You will now see how this collaboration may be illustrated in a sequence diagram. As in Figure 5, the identifiers `pupil3`, `pupil4` and `pupil5` are used. For the purposes of this section assume that the `Pupil` objects are iterated over in the order `pupil3`, then `pupil4`, then `pupil5` (the actual order is not important, and indeed may vary between executions).

Exercise 6

Figure 11 shows the first of the collaborations between `form2` and its `Pupil` objects. Complete the diagram to show the collaborations between `form2` and `pupil4` and between `form2` and `pupil5`. (You will need to refer to Figure 5 for the birth dates.)

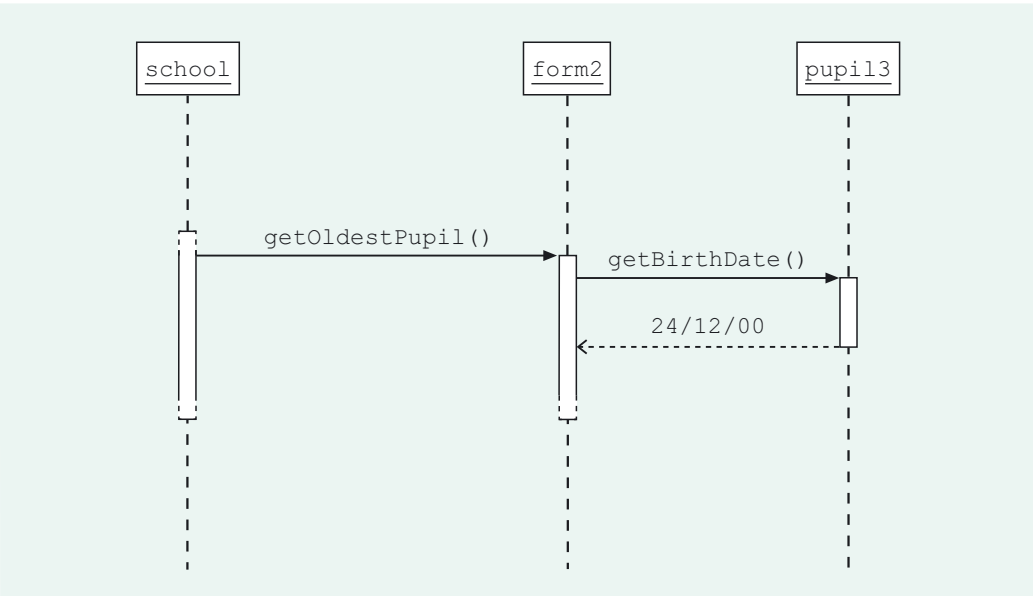


Figure 11 `form2` collaborates with `pupil3`

Discussion

Figure 12 shows the completed sequence diagram.

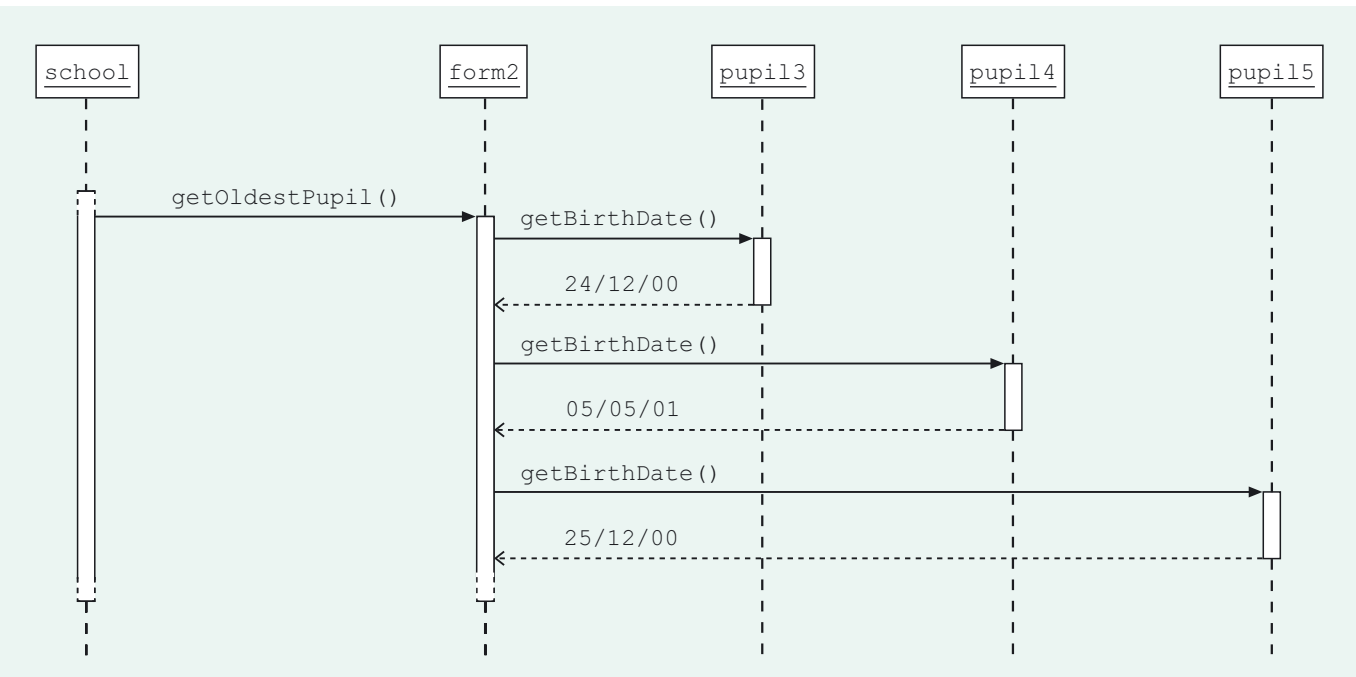


Figure 12 `form2` collaborates with each of its `Pupil` objects

SAQ 3

The method `getBirthDate()` of the `Pupil` class is as follows.

```
public M256Date getBirthDate()
{
    return birthDate;
}
```

Does a `Pupil` object collaborate with any other object when executing its `getBirthDate()` method?

ANSWER.....

No. When a `Pupil` object receives a `getBirthDate()` message it simply returns the value of its `birthDate` instance variable.

Figure 13 shows a sequence diagram illustrating the complete message sequence involved in finding the oldest pupil, starting with `userInterface` sending the message `getOldestPupil(form2)` to `school` and finishing with `userInterface` receiving the object `pupil3`.

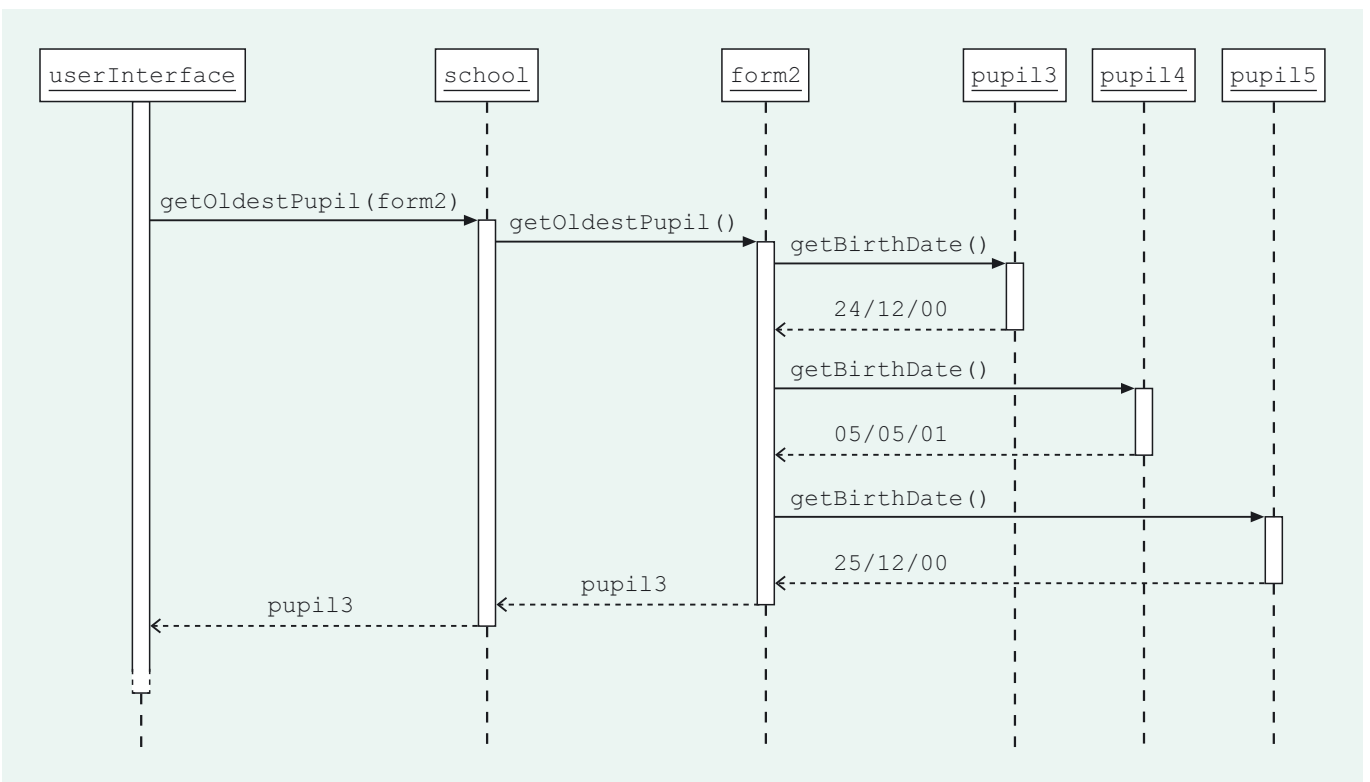


Figure 13 The complete message sequence responding to the request for the oldest pupil in Form 1b

In the next exercise you will practice what you have learnt about object and sequence diagrams, by looking at exactly the same task (locating the oldest pupil in a form) but with a different scenario, i.e. a different combination of objects and links.

Exercise 7

(a) The form named Form 1c has one pupil in it: her name is Sophie Webster and her date of birth is 04/11/00. Ms Yingjie is the teacher of this form. Draw an object diagram, using the identifiers `form4`, `pupil6` and `teacher1`, to illustrate the objects that correspond to these real-world entities.

(b) A pupil named Craig Harris enrols into Form 1c. His date of birth is 02/07/00. Extend your object diagram to illustrate the `Teacher`, `Form` and `Pupil` objects involved, choosing a suitable identifier for the additional object.

(c) Suppose that a user of the School System selects `Form 1c` in the user interface. Draw a sequence diagram to illustrate the sequence of messages and message answers that pass through the system for this scenario, resulting in the `Pupil` object corresponding to the oldest pupil in Form 1c being returned to the user interface.

Discussion

(a) The object diagram illustrating the scenario is as follows.

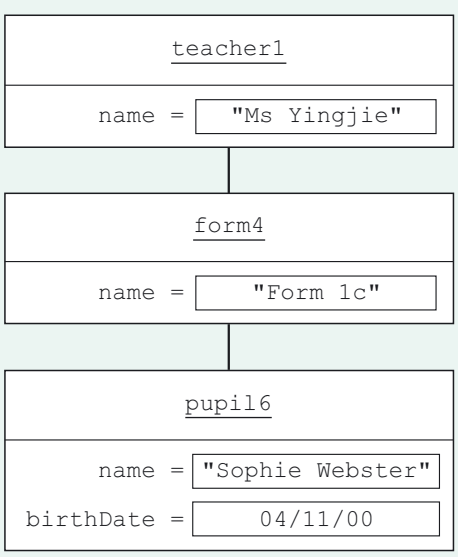


Figure 14 Object diagram illustrating Form 1c, its teacher and its pupil

(b) In our updated object diagram we have used the identifier `pupil7` for the additional object. You could have used any identifier that was different from the ones that have already been used in this unit.

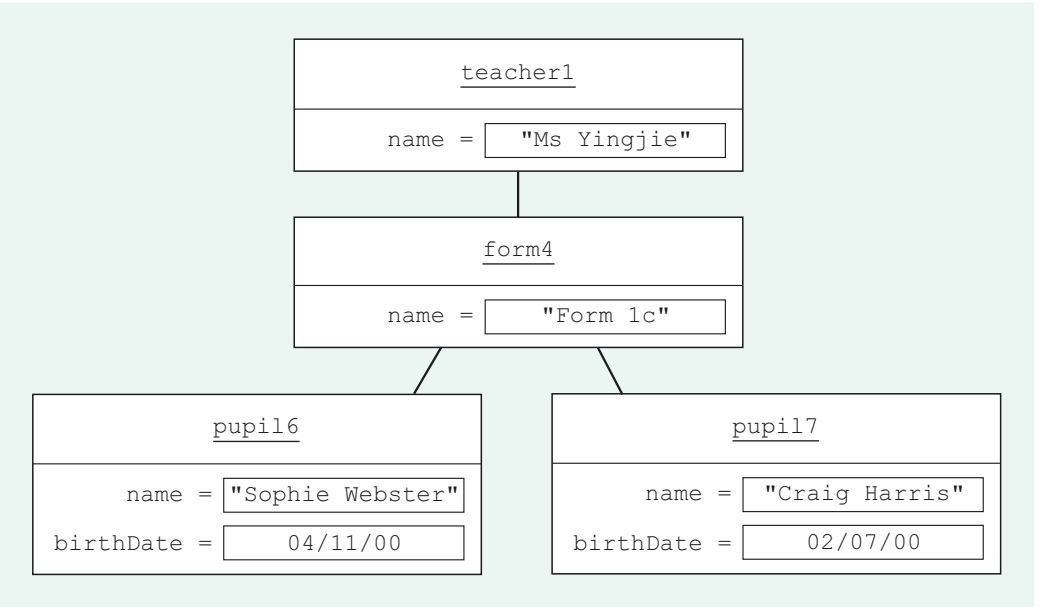


Figure 15 A new pupil in Form 1c

(c) A sequence diagram, showing the oldest pupil from Form 1c being obtained for this scenario, is shown in Figure 16. Note that the order in which the `Pupil` objects are sent the message `getBirthDate()` does not matter.

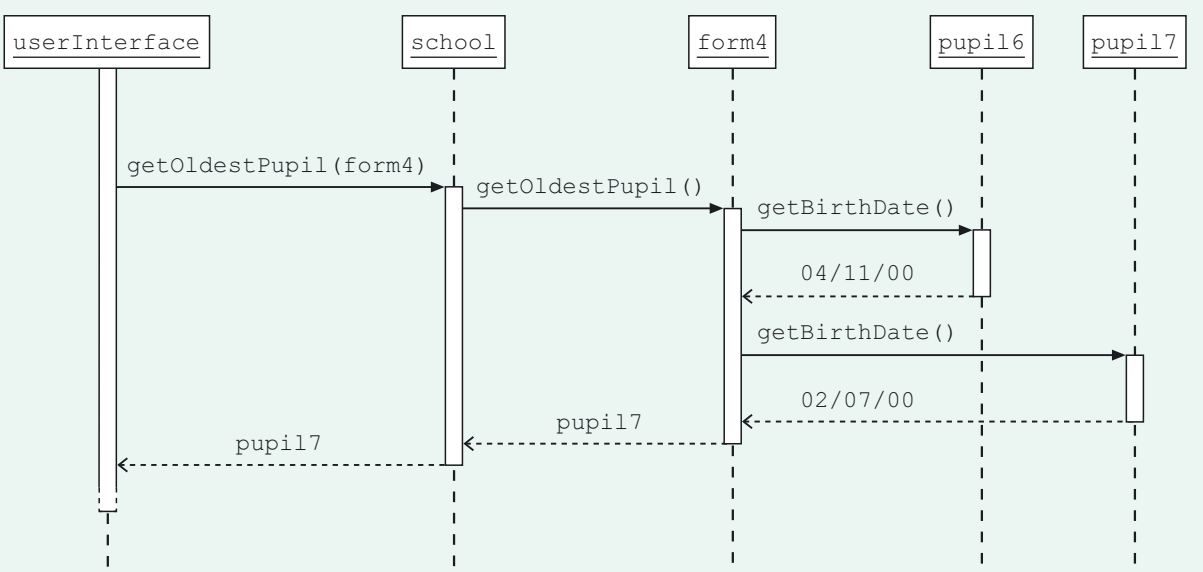


Figure 16 Getting the oldest pupil in the different scenario

In this section you have explored how objects collaborate when the School System carries out one of its tasks. Sequence diagrams were introduced as a way of picturing the sequence of messages involved for a particular scenario. The sequence was deduced by considering the system code.

Sequence diagrams have other uses than simply representing how an existing system works. As you will see in the next section, and again later in the course, they can be used as tools in *designing* software, allowing you to sketch out possibilities for how a system *might* work.

4 Development phases and models

Determining the oldest pupil in a form involves, as you have seen, collaborations between several objects. As this is just one of a range of tasks the School System has to be capable of it would appear that developing the software must have been a fairly intricate process. It was, but because the project was carefully broken down into a series of smaller chunks, which were worked on separately but then linked back together at the end, the developers (who were the M256 course team) were able to keep the complexity under control. As in many areas, decomposing a complex problem into a number of simpler subproblems is a powerful strategy in software development.

In this section you will learn about:

- the main software development phases that help software developers progress from a description of the requirements by the **client** (the person or people commissioning the software) to a deliverable working system;
- the concept of software models;
- a language for depicting software models called the UML.

This use of the term 'client' should not be confused with 'client' as an object in a client–server collaboration. However, the relationships are analogous: the client here is requesting a service of the developer.

4.1 Object-oriented software development

In developing the systems in this course, you will be working towards an implementation in Java. Java is one of a number of programming languages described as *object-oriented* (others in common use including Smalltalk, Eiffel and C++), and several features of Java, such as the following (which should be well-known to you) are actually common in all object-oriented programming languages.

- Classes – blueprints which define the common attributes and operations that a group of objects have in common.
- Inheritance – the definition of one class as a special kind of another class.
- Messages – communications sent to an object, causing a corresponding method to be executed.
- Data hiding – the protection of an object's implementation details by preventing other objects directly accessing its code and state.
- Polymorphism – the capability for objects of different classes to respond to the same message in a manner appropriate for each class.

A central aim of object-oriented software development, whatever language the software is to be implemented in (the **target language**), is to define classes which will result in a set of objects collaborating appropriately to achieve the tasks required of the system.

Most of the ideas introduced in M256 are therefore applicable in any object-oriented language. In fact, as you will learn, even when using a particular target language (Java, in this course), there are benefits in carrying out much of the software development in a language-independent way. Language-independent development ideally means that it should be easier to implement the software in another language if necessary, but also that the focus of the development can initially be on the bigger issues, before it becomes immersed in language-specific detail.

4.2 Breaking down the task

Humans achieve many complicated tasks through following, consciously or unconsciously, a process of smaller, more manageable 'planning' stages. Consider the construction of a building. A process involving several levels of planning and modelling (creating different architectural plans, for example) is carried out to organise the construction engineers' thoughts (and those of their clients), before any part of the building is actually constructed.

SAQ 4

Consider the task of going on holiday. How might this be successfully organised through a succession of stages, each planning some aspect of the trip?

ANSWER.....

You might begin by thinking 'Let's take a winter break in the sun'.

Then you might visit travel agents, collect brochures, go online and consider possible dates and costs.

Next you might take decisions about where and when to go, make reservations and book leave.

Finer details are then sorted out, such as how to get to the airport, what time to get up on the day you leave, and who will feed the cat.

Finally the plan is put to the test and you set off on holiday.

The task of creating software similarly benefits from being accomplished through a systematic succession of smaller, interlinked stages, or **phases**, each consisting of different activities, and each building on the previous phase. The task of going from a description of software requirements to a collection of software objects sending messages to one another is a large and complex one, which can very easily go wrong (or may not even be possible at all) if attempted in one step. The task needs to be broken down into smaller phases that are easier both to manage and to carry out.

Painting and programming

> There is no essential difference between the way in which a painter plans and 'implements' a picture and the way in which a programmer plans and implements a program....
>
> [In a recent exhibition]...there was one vast, unfinished canvas that revealed exactly how (the artist) had worked on it. He had sketched in the major structure, some parts completely finished, others only partly painted – exactly how a good programmer writes a program....The processes of abstraction, visualisation and realisation are the same, just the application area is different.

Excerpt from Marshall (1992).

In software development the initial focus is usually to get an overview of the required system. That is the developer concentrates on planning the overall structure of the system and not on smaller details. As the project progresses, more detailed aspects of the software are considered. Thus, the production of what will eventually be a complex system is made manageable by following a development process that considers appropriate levels of detail at appropriate times. This can be thought of as moving through different levels of abstraction as more and more detail is added to the plans.

A systematic development process also has the advantage that more than one person can be involved. If there is good communication between those involved, meaning not only that they talk with one another but that the scope and results of each activity are clearly set out, then allocating people to different phases enables the distinctive skills of individuals to be combined.

The object-oriented software development phases you will learn about in M256 can be described as follows.

- **Requirements specification**. This involves eliciting and analysing the client's wishes, in order to produce a detailed and complete specification of the tasks required (i.e. the required functionality) of the system.
- **Developing a conceptual model**. Here the requirements are analysed to determine the classes and connections between them that appropriately model the key concepts in the real-world area the system is being written for. Hence this stage defines an initial structure for the system.
- **Developing dynamic models**. Here, models of the interactions among objects, which will achieve the tasks required of the system, are designed and compared.
- **Developing a user interface**. This phase involves both design of the user interface and determination of how it will communicate with the core system.
- **Detailed design and implementation**. In this phase decisions are taken as to which existing classes can be reused and what programming constructs are appropriate, and the actual code is written.
- **Testing**. This involves not just testing the final product but testing at each stage to ensure that the phases of development are consistent and complete with respect to each other, and also consistent and complete with respect to the requirements.
- **Maintenance**. The aim of the maintenance phase is to keep the system working to the satisfaction of its users. It may include tasks such as:
  - fixing emerging problems;
  - fine-tuning the system to improve its performance;
  - enhancing the system by adding extra facilities.

You may be surprised to find that a software development project is normally not considered to be complete once the system is up and running and doing the job required! This is where maintenance starts.

Traditional software development phases

Traditionally, software development is considered to involve the following phases.

- Requirements specification. As above.
- **Analysis**. Involves analysing the specified requirements and expressing, in computing terms, *what* the system should do.
- **Design**. Involves deciding *how* the system will meet the specified requirements.
- **Implementation**. Involves translating the design into program code.
- Testing. As above.
- Maintenance. As above.

However, when following an object-oriented approach to software development the distinction between analysis and design becomes blurred. While it is still important to distinguish between *what* the system has to do and *how* it is to be achieved, the activities of analysis and design can be quite closely interleaved. In analysing the real-world tasks the system has to carry out, it is natural to think in terms of objects, because the structure of object-oriented software often resembles the real-world entities the software is concerned with. Thus, at an early stage the developer will consider not only what tasks the system is required to carry out, but also what objects will participate in the achievement of these tasks.

A significant aspect of software development is the creation of models, which we will discuss in the next subsection.

4.3 Models

A **software model** is a plan: an illustration or description of the software, or of part of it, which emphasises certain aspects and omits others (i.e. it is an abstraction). A good analogy is a map of the London Underground, used by travellers moving between stations in the underground railway system. Such a map is shown below.

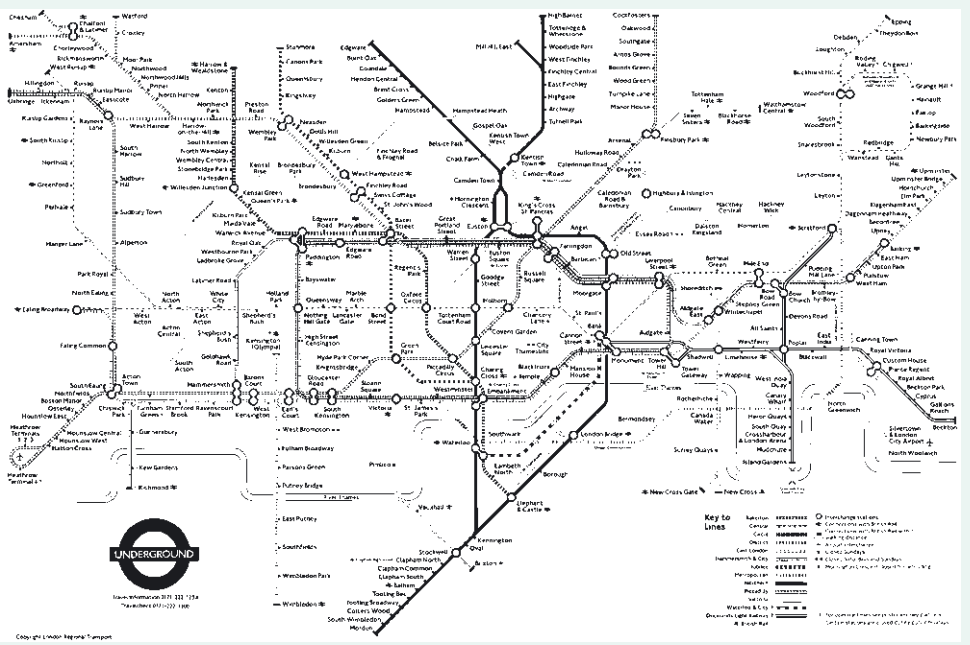


Figure 17 Map of the London Underground

The map is a representation of the London Underground system: it does not show the precise geographical layout of the lines, or how the tunnels are constructed, and it does not show the location of toilets or where tickets are collected. The map is an abstraction and what it does show is a stylised description of the topological relationships between stations and connecting lines – the only information required by underground travellers to plan their route. It is a *model* of the underground system. Any information about ticket machines, toilets, and so on, would only clutter the map and make the task of finding a route through the underground system more difficult.

Similarly, the models used at a certain point in software development highlight information that is relevant at that point and suppress information that is irrelevant.

Figure 18 shows a simple model that we (the M256 course team) created during the development of the School System. It relates to the requirement for the system to provide the name of the teacher with the most pupils in their form.

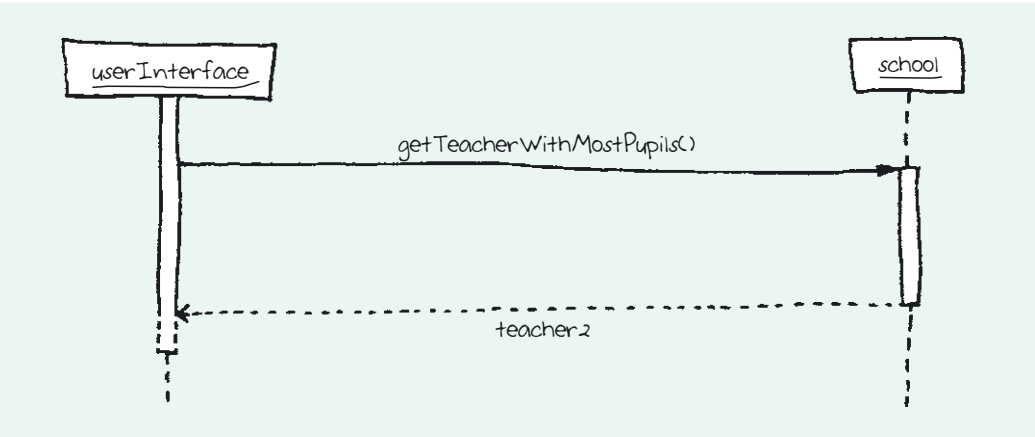


Figure 18 Getting the teacher with most pupils

You will notice that Figure 18 is just a sequence diagram, no different in style to those introduced earlier in this unit. There is however a significant difference between how you have previously used sequence diagrams, and how a diagram such as that in Figure 18 is used during the development of a system. Whereas you have previously used sequence diagrams to illustrate how an already operational system works, in a software development project they are generally used as modelling tools to explore and plan design possibilities for the system. In other word, sequence diagrams illustrate ideas for how the future system *might* work.

In creating the sequence diagram in Figure 18 the developers were expressing the idea that, as part of what the system does to obtain the teacher with the most pupils, the `school` object could receive a message `getTeacherWithMostPupils()` from the `userInterface` object and respond by returning a `Teacher` object. The sequence diagram is a model that emphasises a collaboration between `userInterface` and `school`, but neglects details such as how the method corresponding to the message `getTeacherWithMostPupils()` could be coded.

Exercise 8

Figure 19 is a further sequence diagram, again related to the requirement of getting the name of the teacher with the most pupils. Outline, in words, what the diagram shows happening in terms of messages and message answers.

Discussion

The diagram shows `school` sending several messages as part of its response to receiving the message `getTeacherWithMostPupils()` from `userInterface`. It sends a `getFormSize()` message to several `Teacher` objects, each of which responds with an integer.

A reasonable guess, given the task in question, would be that *all* the `Teacher` objects in the system are sent this message by `school`, and that each of them responds with the size of the teacher's form. The sequence diagram would have to be augmented with some additional information for these facts to be clarified.

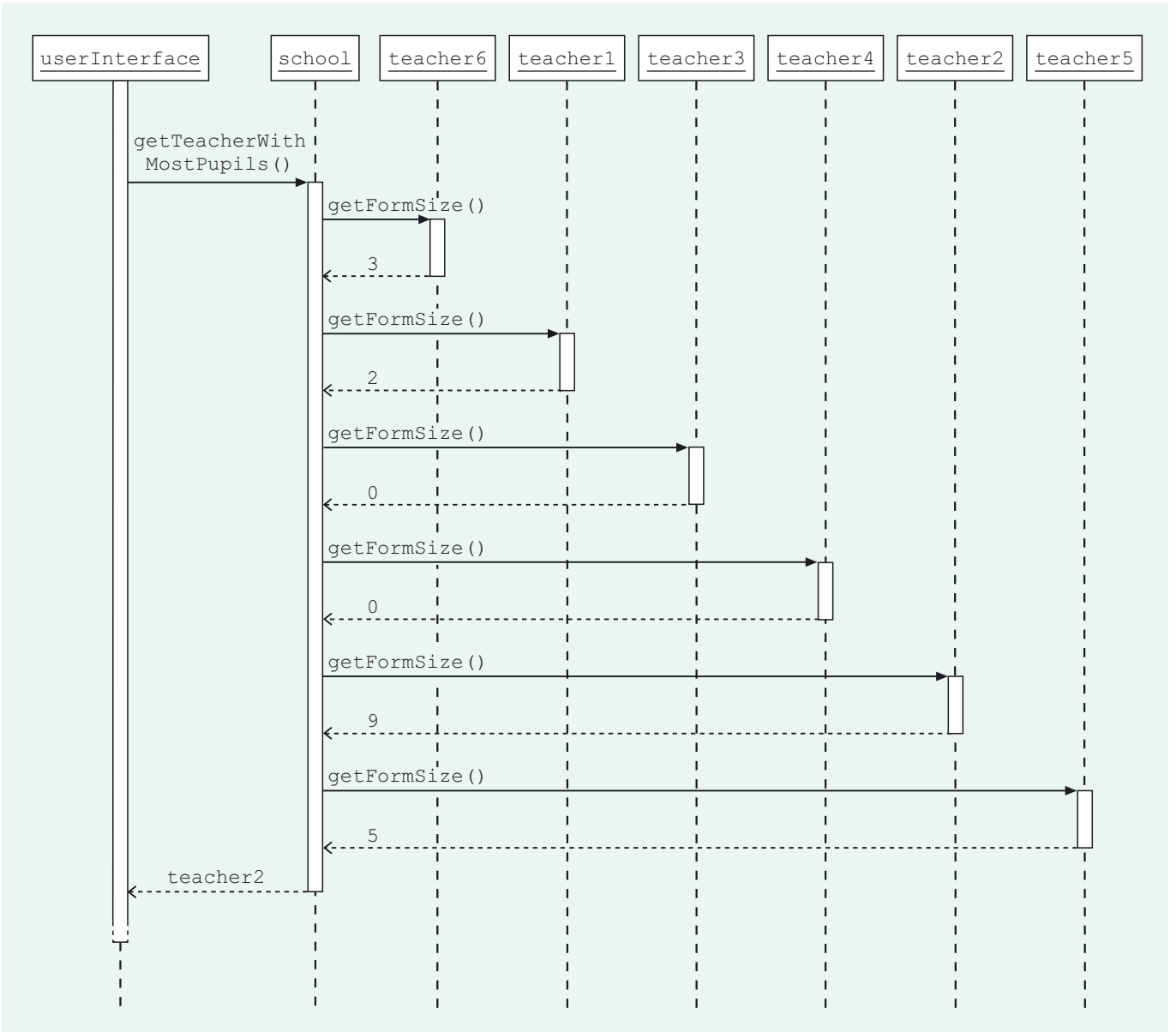


Figure 19 Getting the teacher with most pupils: the `Teacher` objects get involved (please note that the School System is NOT implemented in this way)

In fact, the M256 course team decided on another approach for this requirement.

Had the developers decided to pursue the ideas expressed by the sequence diagram in Figure 19, they would have proceeded to write corresponding method code. The method `getTeacherWithMostPupils()` in the class `SchoolCoord` (the class of the object `school`) would have been written so that it carried out the following.

1 Iterated over all `Teacher` objects, sending each a message `getFormSize()`.

2 Calculated, from the answers, which was the largest form size.

3 Returned the corresponding `Teacher` object.

In the preceding paragraph we effectively described in words how the system could carry out a task. So why do software developers use models, and why models in the form of diagrams?

4.4 Modelling and diagrams

In a systematic approach to developing a system, models of the system are produced at appropriate levels of abstraction, with increasing detail as development progresses. On an individual basis this modelling helps organise thinking about what might be a very complex task. In the context of a team working on a project, using models promotes the sharing of ideas and the successful division of tasks. For example, the design and the implementation (the actual programming) might involve different people. The designer can hand over to the programmer a set of models representing the part of the system to be implemented. The designer need have no knowledge of the precise implementation details that the programmer may introduce; similarly the programmer need not be aware of how the designer came up with the designs. The models represent the information they need to share, and therefore constitute an important part of the communication between them.

Models also have an important role in exploring different designs. Later in the course you will negotiate your way through a maze of different design possibilities by drawing and analysing diagrams representing different models.

Expressing a model using a diagram has several advantages over textual descriptions.

1 A diagram is a concise, abstract form of communication amenable to emphasising certain features and suppressing others.

2 A simple diagram can often be understood by someone inexperienced in computing (such as the client commissioning the system, for example), whereas a textual description might not.

3 In an object-oriented approach, objects begin to be identified right from the start of a project. This means diagrams involving these objects can evolve seamlessly as they incorporate increasing levels of detail through the development process. In other words, the same kinds of diagram can be used throughout, lessening the cognitive load on the developer.

The M256 units mostly present electronically prepared diagrams. As mentioned previously, when using diagrams as an exploratory tool you should feel free to sketch them in any convenient form – pen and paper is often fine. However, when a diagram is used as a blueprint for how a system is subsequently to be developed (i.e. handed over from designer to programmer) then the need for precision and clarity usually necessitates electronically prepared diagrams. Similarly, it will be necessary for you to produce electronically prepared diagrams for your answers to TMA questions.

4.5 The UML

You have already met some of the models used in M256 – object diagrams and sequence diagrams. The models in this course are based on the **UML (Unified Modeling Language)**. The UML is an example of a **modelling language** based on diagrams. A modelling language specifies how models should be constructed so that the meaning of the model is unambiguous. It is not a *method* for developing software, but a way to produce models that can be used in different methods of software development.

Note the American spelling of modeling in UML.

Think of a language for human communication. It has:

- a vocabulary (the elements of the language);
- a grammar (the valid ways in which its vocabulary can be combined);
- semantics (what each valid combination of vocabulary means).

Similarly a modelling language has *modelling elements* (particular styles of boxes and lines, for example) and *conventions* (that prescribe, for example, what combination of elements in a diagram is valid, and that allow the meaning of a valid combination of modelling elements to be interpreted). For example, in the UML, two boxes of a particular kind with a particular style of line between them, as in Figure 20 (a), can be a valid diagram (it is an object diagram), whereas two lines emanating from a box, as in Figure 20 (b), cannot.

Figure 20 (a) A valid UML diagram (b) An invalid UML diagram

A valid diagram represents a meaningful aspect of a system; an invalid diagram does not. Thus a modelling language such as the UML enables the construction of meaningful diagrammatic models of a system.

Using the UML correctly means adhering to the current **UML standard**, that is, the currently accepted specification of what is valid UML and how it should be used. UML standards are set by the **Object Management Group (OMG)**.

The OMG

The OMG is a vast international consortium of computing companies which sets standards across the parts of the software industry concerned with object-oriented technologies. Its purpose is to facilitate communication within the computing industry, and to promote interoperability of object-oriented products from different companies. Any company can sign up to the OMG and thereby potentially influence the development and maintenance of standards. Compliance with OMG standards is generally seen to be an advantage – in fact something a company will advertise as an attractive feature of its approach or products since it implies ease of communication with other companies and interoperability with other products.

The rise of the UML

As object-oriented programming grew throughout the 1980s and 1990s, so too did the number of modelling languages used for discussing and recording software development. From the proliferation of modelling languages one could be selected, or adapted, to suit a particular project and the people working on it. Those intimately involved in a project understood the kinds of model used, but there was no guarantee that anyone else would. Someone wanting to reuse part of the design at a later stage (for example, to implement the system in a new programming language) may have had the overhead of first getting to grips with an unfamiliar modelling notation. Reusing, and even simply discussing designs, was made difficult by not having a consistent and shared means of describing them.

The UML was the result of an attempt in the late 1990s to establish a standard modelling language and rules for using it. Eminent software developers worked together to unify the confusing variety of existing modelling languages, resulting in proposals to the OMG for a single modelling language: the UML. A UML standard was then set by the OMG that specified diagram elements and notation, how they could be combined, and what they meant. The UML standard is evolutionary, in the sense that there has actually been a series of standards, each building on the previous as software developers place new demands on models. At the time of writing the current UML specification is UML 2.0.

The UML is a vast and, in places, highly complex language – in this course you will meet a very small subset of its diagrams. This is actually typical of a software project; although most professional developers have a general understanding of the expressiveness of the UML, most projects will only require them to work with a limited range of diagrams.

UML progress

The *Software Development Times* in August 2004 reported research findings from a survey of over 300 development managers, which included the following points.

- More than two-thirds of development managers said that the UML was used to some extent within their organisation. About 20 per cent said that the UML was used for all projects; 58 per cent said that it was used for some projects.
- The most common reason stated for using the UML was that it improves communication within a project team. Other popular reasons were because the UML allows software to better meet requirements; because applications written with the UML are easier to maintain; because the UML allows software to be built more quickly; and because software written with the UML has fewer defects.

Zeichick (2004)

Although the UML is generally acknowledged to have made significant contributions to software development, it is also accepted that its necessary rigour makes strict adherence rather cumbersome. In particular, when using diagrams to explore different design possibilities, the UML is often not strictly adhered to. Developers using the UML for informal peer discussions will not see the benefits of, for example, remembering to use the right kind of arrows all the time, and they may annotate, or otherwise alter, a UML diagram to suit their own needs.

So long as the main features of diagrams follow the UML, small variations tend to be unproblematic. This use of **UML-type diagrams** (i.e. ones that vary slightly from the standard) rather than **strict UML diagrams** is generally considered acceptable, and in this course we adapt the UML similarly. The course team, in common with other developers, use what we find useful and adapt it to suit our circumstances. You should not feel uncomfortable about this. If your future work or studies mandate strict use of the UML then this course will have given you a firm foundation on which to explore the standards further.

Exercise 9

A group of friends who have some experience of object-oriented software development are working together to create a software system for managing their local football league. The system will undertake various tasks, including providing information about each team (for example, who the manager is) and about matches that the teams play amongst themselves in a season (who plays who, who has won the most matches etc.).

(a) State the main advantages of the friends following a planned development process.

(b) Give two reasons why it would be a good idea for them to use the UML.

Discussion

(a) The advantages of following a planned development process are, firstly, that the complexity of the system would be easier to handle, and the development made simpler. Secondly, the planned process would allow workload to be shared, and skills put to best use, by allocation of different people to different tasks.

(b) Any two of the following are valid reasons for using the UML.

- The use of models based on the UML means that the group would have a consistent and unambiguous means of communication.
- Using the UML would enable analysis of different possible plans for the system.
- As the group would be following an object-oriented development process then essentially the same kind of diagram could be used throughout, reducing the number of different types of diagram involved and simplifying the process.
- UML diagrams are useful for producing diagrams at an appropriate level of abstraction (allowing detail that is irrelevant at a particular point to be suppressed).
- UML diagrams have a better chance of being understood by people other than the diagram's creator. Some diagrams can be understood by non-computing specialists (team managers, for example, might need to know about some of the plans for the system).
- The operational system will be more readily understandable and reusable if UML diagrams describing the system are available.

In this section, you have learnt about the phases of software development that will be used in this course. You also learnt how models are used in these phases and, in particular, how and why the UML is increasingly being used as a modelling language for object-oriented software development.

The next section briefly explores the relationship between the phases of software development and software development methods.

5 Software development methods

Earlier you were introduced to the concept of developing software in phases, each building upon the previous phase. This section gives an overview of how the phases of software development may be combined to form a software development method.

5.1 What is a software development method?

Before discussing what is meant by a software development method, it will be helpful to review briefly what has been learnt about the phases of software development.

In Section 4 we introduced the following main phases of object-oriented software development:

- requirements specification;
- developing a conceptual model;
- developing dynamic models;
- developing a user interface;
- detailed design and implementation;
- testing;
- maintenance.

SAQ 5

What is each of the following phases concerned with?

(a) Developing a conceptual model.

(b) Developing dynamic models.

ANSWER.....

(a) Developing a conceptual model is concerned with analysing the requirements to determine what classes, and connections between those classes, appropriately model the key concepts in the real-world area the software is being written for.

(b) Developing dynamic models is concerned with modelling the interactions among objects that will achieve the requirements (i.e. the tasks required of the software).

It is most important to appreciate that there is no implication that the phases *must* be undertaken in a linear fashion, with each one completing before the next starts. On the contrary, many different permutations are possible. A **software development method** *(*or **method***)* is a particular set of phases, applied in a particular order.

At this point you may be wondering why there is a need for different software development methods. Firstly, there is much debate, and no obvious consensus amongst practitioners and researchers, on the relative merits of different approaches to creating software. Secondly, there can be major differences between software projects, which determine which methods are appropriate. For example, a significant influence on choice of method is the stability of the software requirements, that is, whether they can be fully determined at the outset of the project, and how liable they are to change. The requirements for an embedded system, such as a washing machine controller, or a

safety-critical system controlling a power station, may be well defined from the start, and unlikely to change. In contrast the requirements for a stock control system for a newly established business will change with the changing nature of the business. You will learn that changing requirements can require very flexible development methods.

5.2 The waterfall method

The **waterfall method** is a traditional and idealised view of software development which involves strictly following a sequence of phases. It describes development in which each phase is visited only once, and where each phase is completed before the next begins. Figure 21 illustrates this.

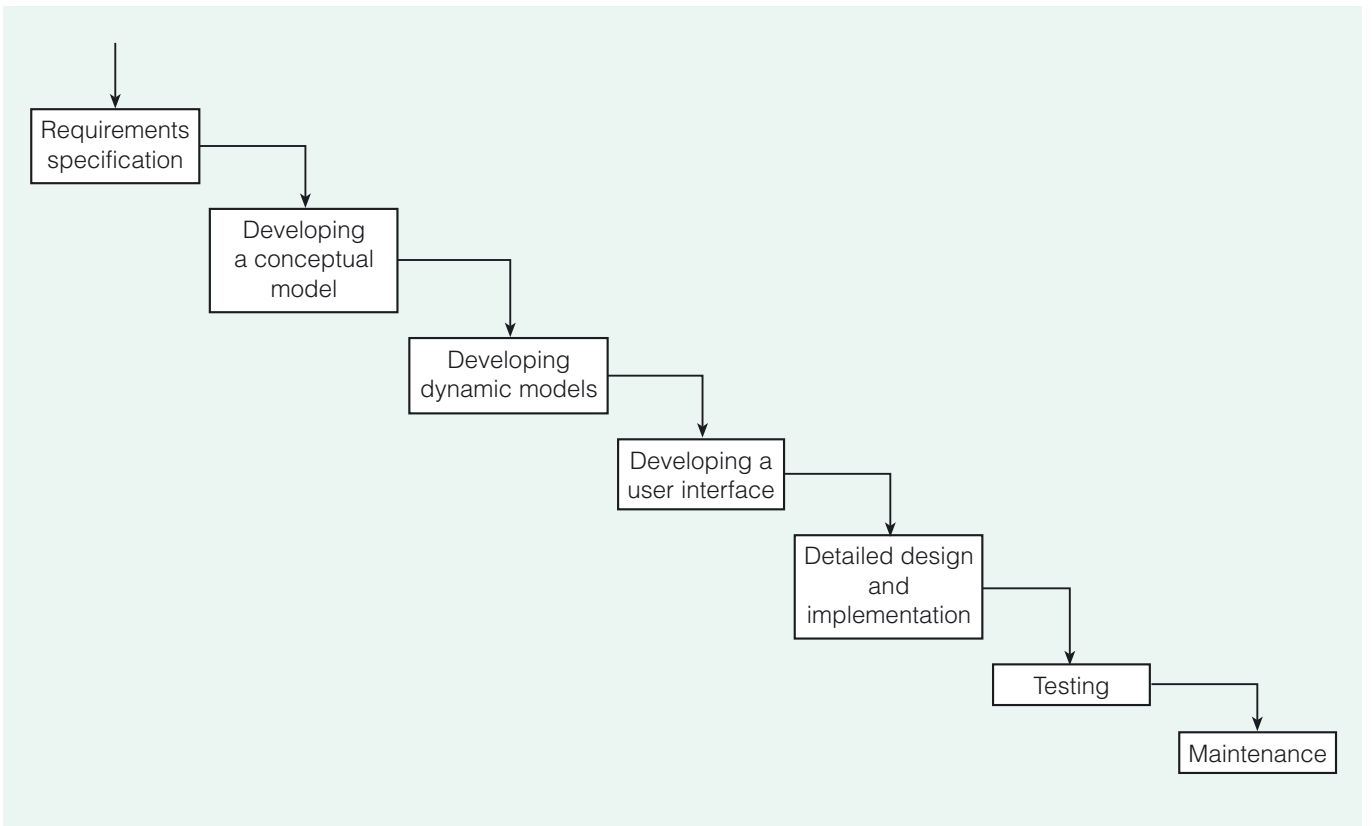


Figure 21 A waterfall model of software development

SAQ 6

Can you think why the waterfall method has been nicknamed the 'throw it over the wall' method?

ANSWER.....

The method is nicknamed the 'throw it over the wall' method since once a phase is completed it is essentially beyond the control of the developers – they may not revisit it.

The waterfall method has some advantages for the management of a project. If there is a set number of phases then developers can at least try to plan in advance for the time and resources required for each phase and then for the entire project. But the method suffers from a number of problems.

1 It does not produce a working system until the end of the project, so the client may not have a good idea of what they are getting until it is too late to make changes.
2 Testing, being at a late stage in the project may be neglected if the project overruns.
3 Errors are likely to be undiscovered until late in the project, meaning that resolving them is rushed or not done at all, or the project is delayed (with the associated problem of considerable costs being incurred).
4 The method does not countenance changes or additions to the requirements as the project progresses, but relies on all the requirements of the system being established at the beginning. This is often unachievable.
5 There is no allowance for the developers to return to a phase to revise earlier decisions.

As you saw at the end of Subsection 5.1, many projects do not start with a fixed and unchanging set of requirements, and most developers do not make perfect decisions consistently. Thus, rigid adherence to a waterfall method is generally unrealistic. Nevertheless, many projects do follow an approximation to it (deviating, for example, by allowing a return from implementation to dynamic model design when a coding problem arises), largely because its predictability aids project management. The term **predictive method** is sometimes used to describe a method largely based on the waterfall approach.

5.3 Iterative methods

Whereas a predictive method is inflexible in the face of change, an **adaptive method** of software development is able to respond to change. *Adaptive* describes ways of developing software, which not only tolerate change (to the software requirements, to ideas in the developers' minds, etc.) but which actually embrace change by building space for it into the schedule.

An **iterative method**, common in object-oriented software development, is one such adaptive method. Phases are repeated in a systematic manner, with each **iteration** (one cycle through the phases) enabling the developers to build on the work completed so far, as well as offering an opportunity for reflection and revision.

A common iterative practice is to restrict the initial development to only a small subset of the requirements of a full system. By designing and implementing just a part of what is required the developer is able to get early feedback from the client and thus reveal more quickly any problems arising from misunderstandings of, or changes to, the requirements. Once this initial part of the system has been implemented satisfactorily, additional behaviour can be incorporated by repeated iterations of the development process until eventually the full system is produced.

Figure 22, overleaf, shows an outline of an iterative method.

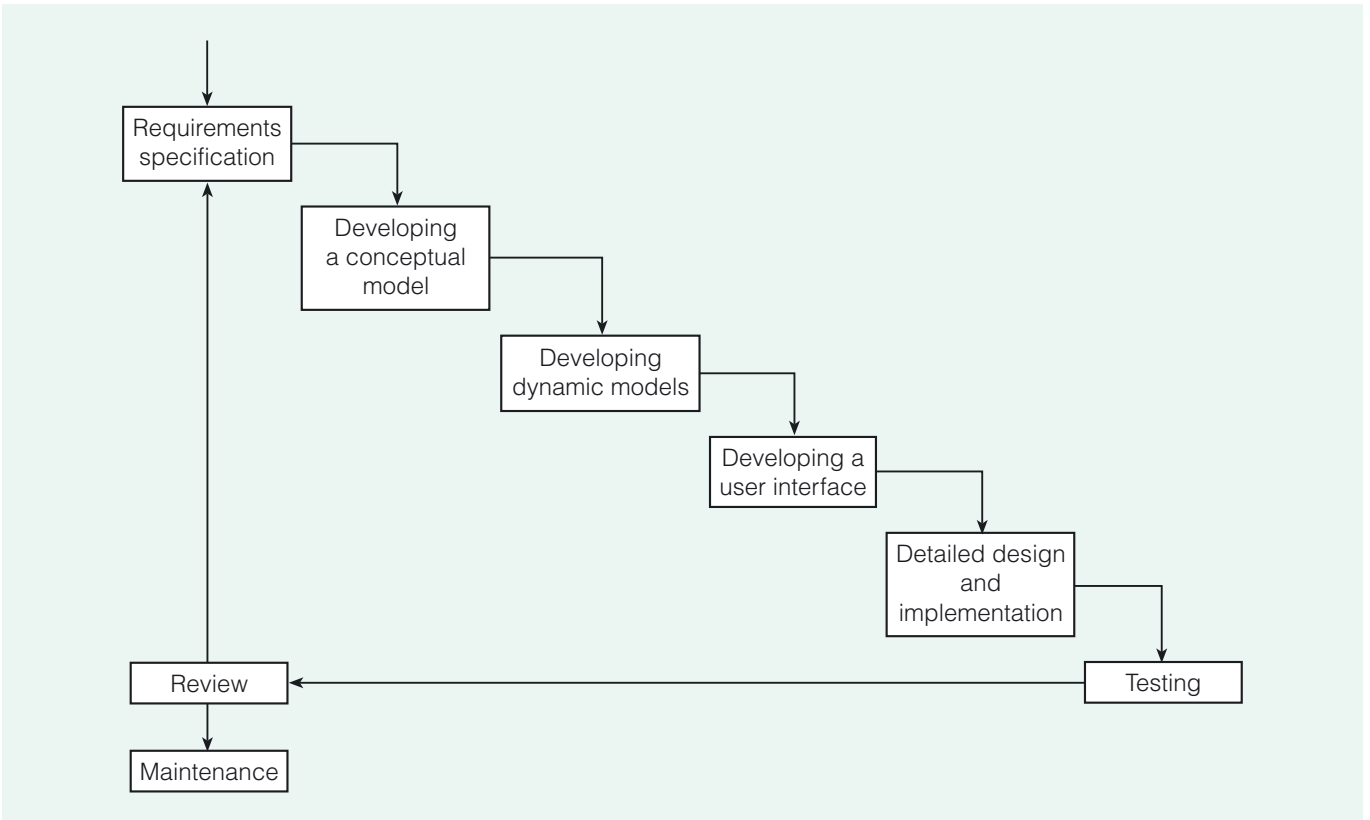


Figure 22 An iterative approach to software development

There are many variations of the iterative approach. A common one is for early iterations to concentrate on getting a satisfactory design of the *structure* of the system before going into the detailed design and implementation and testing phases.

The **review** which is explicitly included within each iteration (see Figure 22) is a point where developers and clients can take changes into account by scheduling them into a future iteration.

SAQ 7

List some kinds of change likely to be identified within a review.

ANSWER.....

Here are some of the kinds of change you might have thought of.

- Changes in the client's requirements.
- Changes to decisions made in previous iterations – about the structure of the system, its design or its implementation.
- Changes to correct any errors from previous iterations.

In each iteration the designs and/or code are tested. Since one iteration builds on another, tests are repeated to ensure that the changes and additions made during an iteration do not damage the previous development.

Within iterative development **prototypes** are often useful. A prototype is an early working version of a system or part of a system, used to test and confirm ideas about what the system is required to do and how best to achieve this. For example, a part of the user interface (with perhaps limited or no actual functionality) may be designed and implemented so that its usability can be analysed and the results fed into the development process.

eXtreme Programming (XP)

Emerging in around 2000, the ideas of eXtreme Programming (XP) challenge the wisdom of developing software through carefully planned phases. XP advocates (at least on certain kinds of projects) concentrating on rapid, prototype-producing, documentation-light iterations of coding and testing. XP is an example of an agile development process, which prioritises the people and styles of teamworking on a project ahead of any process and documentation used.

You will learn more about agile methods in *Unit 14*.

5.4 Software development in M256

There is no single right way to develop software and it is not a primary aim of this course to analyse different software development methods, although from time to time we will indicate different approaches.

In this course, for clarity and simplicity of teaching, we introduce the phases and their activities sequentially, thereby apparently following a simple, linear development process. It is an idealised process since the teaching points would be obscured if we attempted to mirror the complexities of a more realistic one. You might like to think of each of our simple systems as being an initial prototype of a larger, more complex system, and of our progress through the development phases as being the initial iteration of the development of this larger system.

Although M256 does not deal with the maintenance phase in any depth, *Unit 5* looks briefly at what the maintenance phase involves, in order to identify factors in the software development process that enhance the maintainability of a software system.

Figure 23, overleaf, shows a diagram depicting the various phases of development introduced in M256. It also includes the outputs of each phase ('requirements document', 'initial structural model', etc.) which you will learn about as the course progresses. By linking each phase to the unit(s) of the course in which that phase will be studied, the diagram gives you an overview of the structure of the course.

This diagram is also included in the *Handbook*.

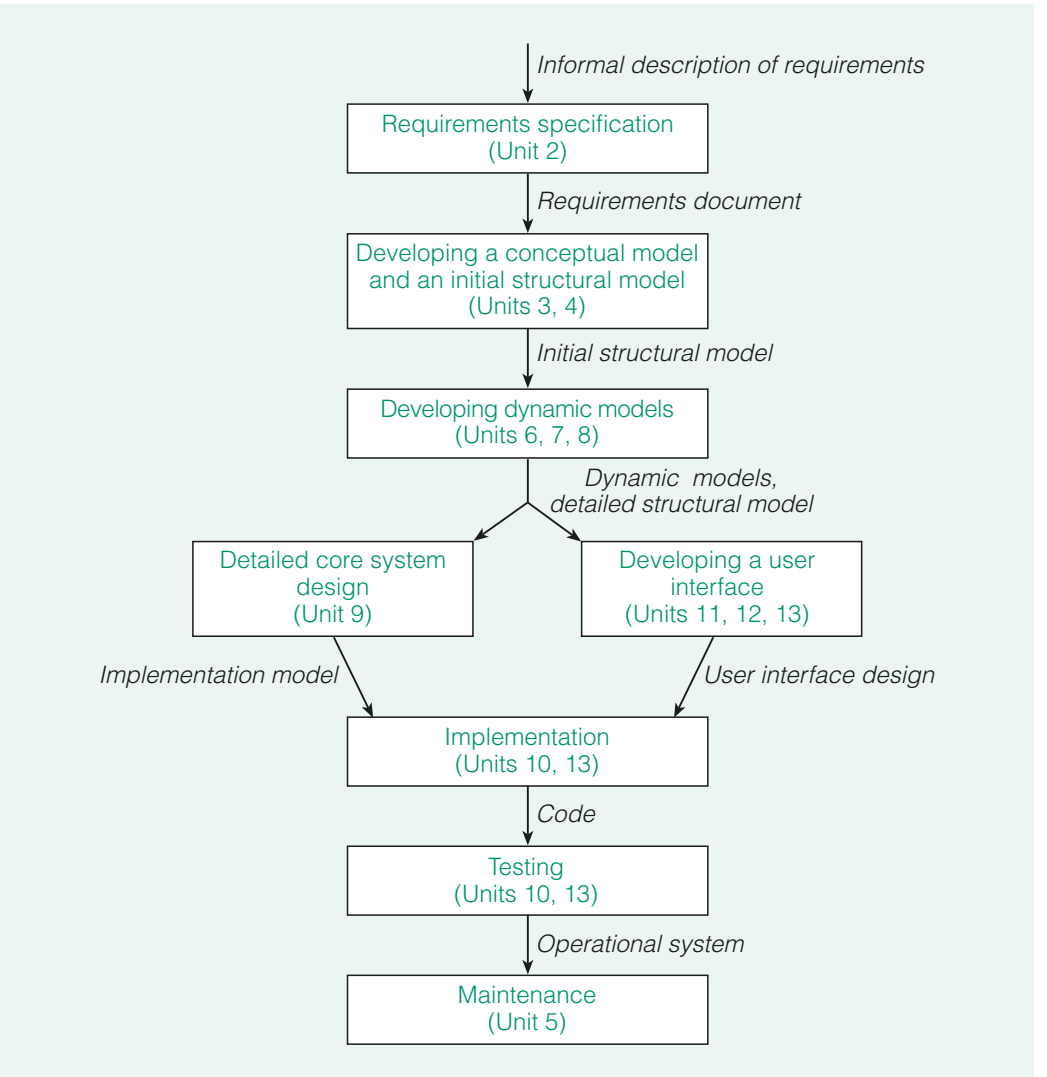


Figure 23 Software development in M256

In this section, you were introduced to the concept of a software development method. You also learnt that there are a number of different types of method, for example predictive (as exemplified by the waterfall method) and adaptive (such as iterative methods). Finally, you saw a diagram that showed the phases of software development you will be studying in this course and how they will be sequenced in the units.

6 Software engineering

Previous sections have introduced you to the phases of software development that will be studied in M256, and to some ways in which they may be combined into a software development method.

In this section, you will learn:

- what is meant by the term software engineering;
- about some of the issues that come into play in the development of large-scale software systems.

You will not be learning about these issues in detail; the intention is to give you the flavour of some of them, to set the work you will do on this course in a wider context.

6.1 Tackling project failure

Consider a software system in widespread use with which you are familiar – a word processing or email application, or even your operating system, for example. Do you have *complete* faith in its ability to perform as it should, without error or delay, enabling you to interact easily with it in carrying out the services it advertises? We think most people would say no. All too often software falls short of expectations.

But why is this? It is not least because developing software involves a complex process of analysing different possibilities and making choices. It is an inventive and therefore challenging activity which offers opportunities for satisfying creativity, but also for disappointment. Disappointing software is often the result of poor software development. The term **project failure** covers situations where a project is unsatisfactory in some way: it might be over budget, it might run over time, or it might result in unsatisfactory software.

Software development can, though rarely, result in a system that completely or almost completely fails. You might like to pause at this point and try to recall an example of a system that has largely failed.

There are many notorious examples. You may have remembered the following.

- The Child Support Agency system put into operation in 2003. Problems with this system resulted in a backlog of millions of pounds of unpaid support payments to single parents.
- The UK Passport Agency problems of 1999, when the introduction of a new computer system resulted in long delays in the processing of passport applications and queues of passport applicants outside the agency's offices.

Common causes of project failure

The UK National Audit Office and the Office of Government Commerce published a list of common causes of public sector project failure in 2004. The points (simplified in places) were as follows.

1. Lack of clear link between the project and the organisation's key strategic priorities.
2. Lack of clear senior management and Ministerial ownership and leadership.
3. Lack of effective engagement with clients.
4. Lack of skills and proven approach to project management and risk management.
5. Lack of understanding of and contact with the supply industry at senior levels in the organisation.
6. Evaluation of proposals driven by initial price rather than long-term value for money.
7. Too little attention to breaking development into manageable steps.
8. Inadequate resources and skills to deliver the project.

National Audit Office (2004)

Systematically developing software is generally considered to be a vital ingredient in a successful project. We noted earlier similarities with the way in which a building is developed; in fact there are similarities with the development of engineering artefacts more generally. A succession of activities is involved, moving from a general description of the software (product) through increasingly detailed designs (engineering blueprints) to the implementation (construction). Because of these similarities to engineering physical artefacts, the term **software engineering** is often used to refer to the wide range of issues connected with carrying out successful software development projects (particularly large-scale ones) using a systematic approach.

The theory of software engineering is a vast one, with substantial industrial practice and academic research behind it. For an idea of its extent, consider the following small sample of research areas.

- Software development methods. For example, which kind of method suits which kind of project?
- Project management. How best to manage the people, tasks, resources and finances involved in a project, so that software of an acceptable quality is delivered on budget and on time?
- Risk analysis. Identifying and managing the possibility of problems occurring in a project. How would a project cope if one of its developers left, for example?
- Testing techniques. For example, what testing strategies are best suited to what kind of project?
- Software quality. What are desirable qualities of software, and how can they be measured and maximised?

The rest of this section introduces you to some other aspects of software engineering relating to **large-scale projects**, to place in context the activities you learn about in the rest of the course. By *large-scale* we mean that the amount and complexity of detail involved cannot readily be handled by one or two people, so that the project is best carried out by a team of people, with different individuals working on different aspects of the project.

6.2 Teamwork

In studying this course you are developing skills and understanding that you can use in developing your own software. However, these ideas are not just useful to individuals working alone, they are also relevant to large-scale software development carried out by project teams within professional software development companies and departments.

When a team creates a software system it is usual for different people to work on different parts of the development. A professional may specialise in a particular aspect of development, concentrating on that aspect in the projects they work on. You may have heard of some of the following job titles, all of which come under the umbrella title of **software developer**.

- A **systems analyst**, who is a technical expert in software systems, analyses the feasibility of a proposed system, evaluating how it may fit into the client's business practices and how those practices may need to adapt with the introduction of the new system.
- A **requirements analyst** is usually involved in the early stages of a project, eliciting and analysing the requirements of a proposed system. They may also become involved in preliminary aspects of design.
- A **designer** works on the design stages of a project. A designer may specialise in certain kinds of design, for example games design or user interface design.
- A **programmer** implements the code, testing small units of code along the way. Again, there are various specialists, such as games programmers.
- A **technical writer** is involved in developing user documentation, such as help files and user manuals.
- A **software tester** tests the software as it is being developed; for example, testing that separate units of code, perhaps written by different programmers, interact appropriately.
- A **project manager** plans and oversees the running of a software development project, from making an initial assessment of the risks involved in the project and allocating people to teams, to having ultimate responsibility for the decisions taken during the project and handing over the software to the client.

It is not uncommon for different software development firms to be commissioned to work on different aspects of a software system. One firm might provide the systems analysts, another the requirements analysts, yet another the designers, and so on. With so many people involved you can see why the project manager figures large in the process!

SAQ 8

Consider the iterative software development method described in Section 5 (Figure 22). Which developers might you expect to be involved at the review stage?

ANSWER.....

A review allows changes to be taken into account by scheduling them into a future iteration. Since these changes can affect the work of any one of the developers, it is quite common that *all* developers are involved in a review. Certainly all those who have been involved in the previous iteration – analysts, designers, programmers and testers – would participate.

6.3 Documentation

Imagine that a team is working on the development of a software system. The team members have different responsibilities: there are analysts, designers, user interface designers, programmers and others. Even with an effective project manager, good communication between the different people involved is a key success factor. Much of this communication is in the form of written documentation.

Project documentation

A programmer will not get far if he or she cannot understand what the designers have decided. Neither will the designers make progress if they cannot understand the work of the analysts. **Project documentation** describes the activities, decisions and outcomes of the different phases of the project.

Project documentation is used *during* a project for communication between developers. It is also a vital ingredient in enabling the operational software to be *maintained* successfully, and allowing aspects of it to be reused in creating new systems. Adapting a system simply by trying to understand and change the code alone is usually doomed to failure or, at best, leads to the production of code that is subsequently unintelligible.

The conclusions reached by each phase of development (e.g. models such as sequence diagrams) obviously should be part of the project documentation. Other kinds of project information may also be relevant: a record of areas of debate and how differences of opinion were resolved, for example. In fact, any information that could potentially be of use to those maintaining the system, or to other developers working on similar projects, is relevant project documentation.

SAQ 9

Why might it be useful for the project documentation to include designs that were considered but discarded?

ANSWER.....

Discarded designs (as well as records of why they were discarded) can be useful to someone charged with modifying the system once it is in operation, or to someone creating a similar system, so that the reason for design decisions can be understood and so that known pitfalls and blind alleys can be avoided.

Program documentation

There are two levels of **program documentation** within software, both of which are vital for its effective use and maintenance.

1 Comments that facilitate understanding of how the software is implemented.

2 Comments that facilitate understanding of what the software does and how to use it.

Comments facilitating understanding of software implementation

You should be familiar with using Java comments to annotate code. Take the `getTeacherWithMostPupils()` method from the class `SchoolCoord` in the School System as an example.

```
/**
 * Returns the teacher with the most pupils in their form.
 * If there is more than one such teacher, returns one of them.
 *
 * @return    a Teacher object, or null if there are no pupils
 */
public Teacher getTeacherWithMostPupils()
{
    int size = -1;
    Form bigForm = null;
           // iterate through all the forms
    for(Form f : forms)
    {
           // if this is the biggest form so far...
        if(f.getSize() > size)
        {
               //...set size to this form's size...
            size = f.getSize();
               //...and set bigForm to reference this form.
            bigForm = f;
        }
    }
    if (size > 0)
        return bigForm.getTeacher();
    else
        return null;
}
```

SAQ 10

Consider the comments in the above code which are denoted by `//`. What is their general purpose?

ANSWER.....

The comments in the code describe how the code works.

There are two main reasons why comments describing how code works can be useful.

1 The programmer can, at a later date, be reminded how the software works: what a particular iteration achieves, the meaning of variables, etc.

2 Someone else looking at the code can understand what was in the original programmer's mind. This may be another member of the team or, at a later date, someone else maintaining or reusing the system.

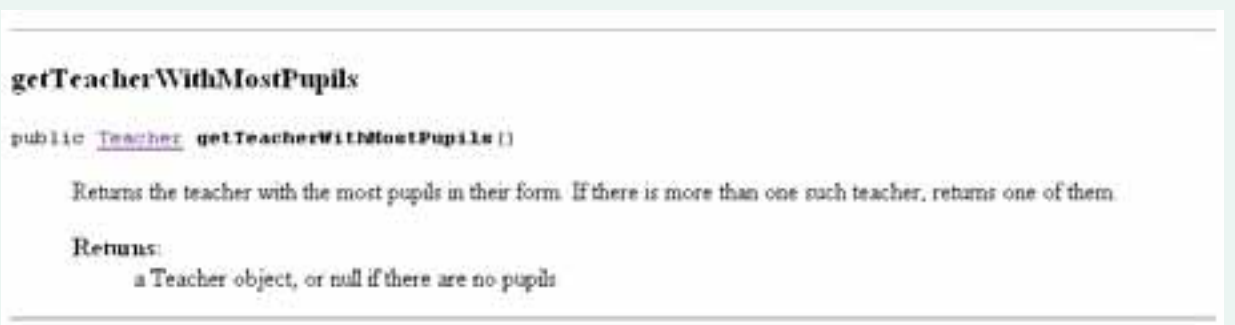
There are other ways of making programs understandable. Programming conventions such as running words together to make a class name (e.g. `SchoolCoord`) are simple contributions. As well as following common programming conventions, a software development company may have its own particular **in-house standards** to which its software adheres. These standards may specify matters from the form of variable names to the naming of packages.

Comments facilitating understanding of software purpose and use

Look again at the `getTeacherWithMostPupils()` method above. The comments denoted with /** and */ describe *what* the method does, as distinct from *how* it does it. If someone else is to make use of a class, they need to know what the class can do for them, in what circumstances it might be used, and how to use it. They may well not need to know *how* the class works, that is they may not need to be able to access the code itself (or even comments on the code).

It is not an aim of this course that you become proficient in using Javadoc.

It is conventional for these comments about what a Java program does, and how to use it, to be written in **doc form**, as here, delineated by /** at the beginning of the comment and */ at the end. The **Javadoc** program provided with a Java SDK extracts doc form comments and uses them to provide a description of a program for public consumption. Here is the documentation produced by Javadoc for the `getTeacherWithMostPupils()` method.



getTeacherWithMostPupils

`public Teacher getTeacherWithMostPupils()`

Returns the teacher with the most pupils in their form. If there is more than one such teacher, returns one of them.

Returns:

a Teacher object, or null if there are no pupils

Figure 24 Javadoc documentation for the `getTeacherWithMostPupils()` method

Exercise 10

You have just seen Javadoc documentation for the public method `getTeacherWithMostPupils()` in the class `SchoolCoord`. As a developer, what elements of a class definition might you *not* want to provide such information about, and why?

Discussion

You might not want to provide information about private elements – private methods, for example. Private methods are intended for use only within the class definition itself; anyone simply using the class should not need to know about them. In fact, the Javadoc program only produces information about public and protected elements of a class definition.

Protected elements will not be discussed further.

Although Javadoc documentation is only produced for public and protected elements, it does no harm for comments on the purpose and use of all elements to be in doc form; such comments being useful for communication amongst people developing the same program.

6.4 Software tools

Software development teams often rely heavily on software tools, sometimes called **CASE (computer-aided software engineering) tools**. Javadoc, which as you have seen is a tool to aid documentation, is a CASE tool. Here are some examples of other kinds of tools, demonstrating the variety available.

Design tools

Design tools provide support for certain aspects of design. A design tool may incorporate a special drawing package which enables the formulation of designs using diagrams. There are many UML-based design tools.

Coding tools

Coding tools provide support for writing and running code. An example of a coding tool is an IDE (integrated development environment) of the kind you use on this course (NetBeans).

SAQ 11

What facilities might an IDE offer?

ANSWER.....

In Section 2 we listed the following.

- A specialised editor for writing and editing source code.
- Facilities for checking syntax.
- Facilities for structuring programs into separate projects, and for creating repositories of associated documents.
- An integrated compiler and interpreter.
- Facilities for 'stepping through' code as it is executed.

Some tools offer integration and automation of elements of design and coding. A tool might enable the user to specify aspects of the design, via UML diagrams for example, and then automatically produce corresponding outline program code. For example, you might produce a sequence diagram which the tool would take as the basis for generating skeletal outlines of methods. The more detailed the design, the more code is automatically generated.

Such CASE tools would appear to significantly reduce the work involved in the production of software. Consequently you might be surprised to learn that some developers prefer not to use them. There are several reasons for this:

1 Developers are forced to describe their designs in a format tightly prescribed by the tool – this may be inappropriate for some projects.

2 The overheads of getting to grips with a necessarily complex tool and working with its idiosyncrasies can be high.

3 Automatically produced code can be less readable and more complex than necessary. Furthermore such code may not adhere to a company's in-house standards.

Exercise 11

Earlier in this unit we described a *UML-type* diagram as one that varies in some minor way from the specification set out in the UML standard.

Suppose a particular CASE tool produces outline code when it is given a design expressed in strict UML. Why would such a tool not generally accept a UML-type diagram instead?

Discussion

A CASE tool is programmed to carry out certain processes (to produce the code) given specific input (a UML diagram). It will not be programmed to deal with other inputs such as even minor variations on strict UML.

Testing tools

There are many different kinds of **testing tool**. A **code-based testing tool** automatically analyses code and produces test cases ensuring that certain aspects of the code (for example, each path through it) are tested. A **test driver tool** executes the software being tested with specified inputs.

JUnit is a tool, incorporated into NetBeans, that assists in the testing of Java programs. It enables the establishment of a testing framework specific to a program, then automatically performs tasks such as initialising objects for testing, and executing specified sets of tests. You will use JUnit later in this course.

The aim of this section was to set the work you will do on M256 in the context of *software engineering*. This is the term often used to describe the development of large-scale software systems, because many of the elements that are necessary to develop such systems in a systematic way are common to the engineering of large physical artefacts like bridges, buildings and aeroplanes.

You were introduced to team work, documentation and CASE tools, which are particular aspects of software engineering.

7 Summary

This unit began by introducing you to the idea of developing software.

Through exploring the objects and collaborations at work in the School System, and using object diagrams and sequence diagrams for illustrations, you learnt about the complexity that can be involved, even in a simple system.

Such complexity is managed by developing software in a systematic, progressive way, with interlinked phases of development and by using models. You were introduced to the phases you will learn about in this course, and to the modelling language, the UML, which enables developers to produce consistent diagrammatic models that are an aid to communication between project members and to documenting the project.

Software development methods – ways of putting the phases of development together – are important when building different types of software system. You were introduced to two kinds: waterfall and iterative methods.

The term software engineering is often used to describe the process of developing large-scale software projects in a way that is similar to engineering any large physical artefact. As a way of setting the work you do on this course in context, you were given the flavour of some of the elements of software engineering: how teams of developers work on a project, including the different team roles, the forms of documentation produced and the variety of tools used to assist in development.

LEARNING OUTCOMES

After studying this unit you should be able to:

- describe and use each of this unit's key terms (summarised in the Glossary);
- explain the purpose of an IDE;
- run and explore a Java program within NetBeans;
- represent objects, and the links between them, using object diagrams;
- identify, by inspecting system code, objects corresponding to real-world entities;
- identify, by inspecting system code, how a link between objects is implemented;
- identify, by inspecting system code, how objects collaborate to perform a task;
- illustrate collaborations using sequence diagrams;
- identify some common features of object-oriented programming languages that render them amenable to similar development processes;
- explain why it is important to develop software systematically;
- outline what is involved in each of the following development phases:
  - requirements specification,
  - developing a conceptual model,
  - developing dynamic models,
  - developing a user interface,
  - detailed design and implementation,
  - testing,
  - maintenance;

- describe the roles of diagrams, models and modelling languages in developing software;
- describe why the UML has grown in importance as a modelling language for software development;
- outline what a software development method is and describe the essential features of waterfall and iterative methods;
- describe some aspects of software engineering; i.e. different team roles, documentation and tools.

Glossary

abstraction A description that focuses on the essential features of a problem and ignores other details.

activation rectangle An element in a sequence diagram that represents a period during which a particular object is active.

adaptive method A method of software development which embraces change by building space for it into the schedule.

analysis In this context, analysis involves analysing the specified requirements and expressing, in computing terms, what the software system should do.

application A program that performs a specific function directly for the user and, in effect, turns the computer into a specialised computer, such as a word processor or web browser.

behaviour (of a software system) The set of tasks the system performs.

call (a method) See invoke.

CASE (computer-aided software engineering) tool A software tool used to help in some aspect of software development.

class A template that serves to describe all instances (objects) of that class. It defines the type of data held by the objects and the object's operations.

client This term has two main meanings in the context of software development: (i) the object in a collaboration which requests a service: (ii) the person(s) commissioning the software.

code-based testing tool A testing tool that automatically analyses code and produces test cases.

coding tool A CASE tool that aids writing or running code.

collaboration One object requesting a service from another object.

collaborator A participant in a collaboration.

core system The part of the system that is distinct from the user interface and that usually contains objects corresponding to real-world entities.

design Deciding how the system will meet the specified requirements.

design tool A CASE tool that aids some aspect of design.

designer A developer whose role is to work on the design stages of a project.

detailed design and implementation Deciding which existing classes can be reused and what programming constructs are appropriate as well as writing the actual code.

developing a conceptual model Analysing the requirements to determine the classes and connections between them that appropriately model the key concepts in the real-world area the system is being written for.

developing a user interface Designing the user interface and determining how it will communicate with the core system.

developing dynamic models Designing and comparing models of the interactions among objects which will achieve the tasks required of the system.

doc form A form of Java comment marked by `/**` at the beginning and `*/` at the end.

dynamic model An illustration of events occurring in a system over time.

identifier A label that is used to refer to an object in a system.

implementation Translating a design into program code.

in-house standards A company's specified standards to which its software must adhere.

integrated development environment (IDE) A software tool which facilitates many of the tasks associated with writing and running programs in a specific language.

invoke (a method) To execute a method's code.

iteration One cycle through the phases involved in an iterative method.

iterative method An adaptive method of software development in which phases are repeated iteratively in a systematic manner.

Java application A program run directly by the Java Virtual Machine.

Javadoc A software tool which extracts doc comments from a Java program and from them provides a description of elements of the program, such as methods, output in the form of an HTML file.

JUnit A software tool that assists with testing Java classes.

large-scale project A project in which the amount and complexity of detail involved cannot readily be handled by one or two people, so that the project is best carried out by a team of people, with different individuals working on different aspects of the project.

lifeline An element in a sequence diagram that represents the time during which an object exists.

link A connection between two objects.

maintenance The phase of software development associated with keeping the system working to the satisfaction of its users.

message A request by one object for another object to provide a service.

method See software development method.

modelling language A specification of how models should be constructed so that their meaning is unambiguous.

object A collection of data and a set of operations that can be applied to the data.

object diagram An illustration of objects and the links between them.

Object Management Group (OMG) A consortium of computing companies which sets standards across the software industry, including the UML standards.

phase A stage of software development.

predictive method A method of software development which is largely based on the waterfall method and which therefore benefits from simplicity of planning, and predictability.

program documentation Comments and explanations designed to assist someone in understanding the implementation of software, what it does and how to use it.

programmer A developer whose role is to implement the code.

project documentation A written description of the activities, decisions and outcomes of a project's phases.

project failure A situation where a project fails to deliver the client's requirements in some way (e.g. in terms of cost, development time, or functionality).

project manager A person who plans and oversees the running of a software development project.

prototype An early working version of a system or part of it.

requirements What is required of the system.

requirements analyst Someone usually involved in the early stages of a project who elicits and analyses the requirements of a proposed system. They may also become involved in preliminary aspects of design.

requirements specification Eliciting and analysing what the client wants in order to produce a detailed and complete specification of the tasks required (i.e. the required functionality) of the system.

review A point within an iterative software development method where developers and clients can take changes into account.

sequence diagram An illustration of objects collaborating to carry out a particular task.

server The object in a collaboration that provides a service.

software See software system.

software developer An umbrella title, referring to someone who takes on one or more of a range of jobs within software development.

software development A planned, phased process, involving modelling different aspects of the software as well as implementing, testing and maintaining it.

software development method A particular set of development phases applied in a particular order.

software engineering A term used to refer to a wide range of concerns connected with carrying out systematic software development.

software model An illustration or description of the software, or of part of it, which emphasises certain aspects and omits others.

software system A program which is large in the sense that it carries out a number of tasks, some of which may be complex.

software tester A developer whose role is to test the software as it is being developed.

state (of a system) The objects, their attribute values and the links between them, which constitute the system at a particular time.

state (of an object) The value of the object's attributes.

static model An illustration of the state of the system, or part of it, at a particular time.

strict UML diagram A diagram that adheres strictly to the UML standard.

system See software system.

system code The source code which, when run, generates the system.

systems analyst A developer who is a technical expert in software systems and whose role is to analyse the feasibility of a proposed system and how it will impact on the client's business practices.

target language The language in which the planned software is to be implemented.

technical writer A developer whose role is to develop user documentation.

test driver tool A testing tool that executes the software being tested with specified inputs.

testing The activities that take place at each phase of development to ensure that the phases of development are consistent and complete with respect each other, and also consistent and complete with respect to the requirements.

testing tool A CASE tool that aids some aspect of testing.

UML standard The currently accepted specification of what is valid UML and how it should be used.

UML-type diagram A diagram that varies in some small way from the strict UML standard.

UML (Unified Modeling Language) A modelling language based on diagrams.

waterfall method A traditional and idealised view of developing software by strictly following a sequence of phases.

References

National Audit Office (2004) *Improving IT Procurement*, London, The Stationary Office, http://www.nao.org.uk/publications/nao_reports/03-04/0304877es.pdf (Accessed 2 June 2006).

Marshall, L. F. (1992) 'They all laughed at Christopher Columbus', in *Proceedings of the Women into Computing 1992 National Conference – Teaching Computing: Content and Methods*, Keele, UK.

Zeichick, A. (2004) *UML Adoption Making Strong Progress* [online], Software Development Times, Huntington NY, http://www.sdtimes.com/article/story-20040815-14.html (Accessed 8 June 2006, registration (free) required).

Acknowledgement

Grateful acknowledgement is made to the following source.

Figure 17: Map of the London Underground. © Transport of London. Transport Trading Limited.

Index